

組込みプログラミング I 最終課題

212 古城隆人

2026 年 2 月 17 日

1 注意

作成した mot ファイルを tera term 経由で書き込もうとしたら、バッファオーバーフローのエラーが出て書き込めませんでした。自作したプログラムにて書き込みを行ってこれまで開発していたため、エラーの原因がわかりません。そのため書き込み用のプログラムと実行ファイルも zip ファイルに同封します。また、Makefile についても独自の構成を行ったため、再現したフォルダ構成を c ファイル_make 用 フォルダに用意しました。配下の fat フォルダのディレクトリで make コマンドでコンパイルとリンクを実行できます。

2 使用した機能

使用した機能と説明を表 1 に示す。

表 1: 機能説明

機能	説明
割り込み 1(MTU2_CH1)	スピーカーの音を鳴らすために使用. コンペアマッチ周期を弄ることにより, 周波数を変更
割り込み 2(MTU2_CH3)	音楽制御用割り込みで, 割り込み 1 と割り込み 2 のコンペアマッチ周期を弄り, 今鳴らす音と次に音を鳴らすタイミングまでの delay を実現
割り込み 3(MTU2_CH4)	7seg 制御用割り込みで, ダイナミック点灯をこの割り込み周期で行っている. また, 表示する内容は今鳴らしている音の周波数である.
割り込み 4(CMT1)	1 秒を取得するための関数. main 関数で一秒周期で実行させる処理があるため, フラグを立てることにより, 1 秒を制御している. また, 画面更新頻度 (fps) を保持する機能も有する.
割り込み 5(MTU0_CH0)	AD 変換のタイミング制御用割り込みで, この割り込み周期は不変で AD 変換モジュールを呼び出している.
ADC(AD0_CH1,CH2)	analog digital converter(ADC) を使用して, ジョイスティックについている可変抵抗器の値 (電圧) をデジタル値で取得している.
DMA(DMAC0 ~ 1)	ダイレクトメモリアドレスで, CPU を介さずにメモリ間転送を実現できる. そのため, 描画する画像を保存している変数から描画用関数への転送や, 描画用変数から TFT への転送に DMA を用いた. これにより, TFT へのデータ送信速度が 100 倍以上速くなっている.
SD カード (SPI mode)	SD カードにゲーム描画用の画像ファイルを保存している. 初期化を行った後に, 変数に SD カードから読み込んでいる.
TFT	ゲームの画面を描画している. ディスプレイに FPGA がつながっており, メモリに書き込む動作を行うことにより, 画面描画を行える.
LED	1 秒周期を確認するために, 0.5Hz で光る LED(1 秒ごとに反転) を実装した.
プッシュ SW	ゲームの操作用に SW4 ~ SW6 を使用している.
スピーカー (SPK)	音楽を鳴らすために使用している.
LCD	ゲームタイトルと操作説明用の文章を LCD に表示させている.
7seg	今再生している音の周波数を 7 セグメント LED で表示させている.

3 ゲームについて

画面が load されたら、タイトルが表示される。その画面で、SW4 を押すことによりゲームがスタートする。

3.1 移動について

プレイヤーは、ジョイスティックにより動かすことが出来る。ジョイスティックの傾きがプレイヤーの速度になるため、たくさん傾けると、移動速度が速くなる。

3.2 ゲームスタート ～ SCORE:200

初期状態で、プレイヤーは左端にスポーンする。この状態で SW5 を押すことにより、スキルを発射する。また、スコアの上昇と比例して、ウルトゲージも貯まっていく。

ウルトゲージがマックスになったときに SW6 を押すことにより、ウルトを発動させることが出来る。このウルトは大ダメージを生み出すことが出来る。そして、敵を倒すことにより、SCORE が加算されていく。敵は画面の右端からスポーンして左方向に移動して、乱数によりスキルを使用する。

3.3 SCORE200 以降

SCORE が 200 を超えると、ボスがスポーンする。ボスは画面の右端の上下を移動している。ボスにはモードがあり、HP が減っていくと、スキルの生成頻度があがったりする。

3.4 一時停止機能

ゲームをスタートした状態で SW4 を押すことにより、一時停止画面が現れる。SW5 が押されると、ゲームがリセットされ、SW6 が押されるとゲームを再開する。

3.5 ダメージについて

敵が生成したスキル (小さい四角) にプレイヤーが触れると、ダメージが蓄積されていく。このスコアは触れた回数のカウントと同じ値であり、いくら加算されてもペナルティや挙動の変化はない。

3.6 終了条件

このプログラムは一度スタートしたらリセットされるまで動き続ける。そのため、終了条件はない。

4 ファイル検索機能について

SD カード内のファイルを検索する関数がある。そのため、保存する file は順不同でも読み込むことが出来る。

5 プログラムリスト

```
1  #include <7080S.H>
2  #include "libmemes.h"
3  //型定義
4  typedef unsigned short uint16_t;
5  typedef unsigned long  uint32_t;
6  typedef unsigned char  uint8_t;
7  typedef signed int     _SINT;
8  typedef unsigned int   _UINT;
9  typedef signed char    _SBYTE;
10
11 // MIDI関連の定数
12 #define MIDI_NOTE_ON    0x90
13 #define MIDI_NOTE_OFF   0x80
14 #define MAX_MIDI_EVENTS 512 // 256から512に増加
15 #define TICKS_PER_QUARTER 480 // 標準的なMIDIのタイミング分解能
16
17 //LCD
18 #define LCD_RS (PA.DR.BIT.B22)
19 #define LCD_E (PA.DR.BIT.B23)
20 #define LCD_RW (PD.DR.BIT.B23)
21 #define LCD_DATA (PD.DR.BYTE.HH)
22 //7seg
23 #define DIG1 (PE.DR.BIT.B3)
24 #define DIG2 (PE.DR.BIT.B2)
25 #define DIG3 (PE.DR.BIT.B1)
26
27 // 音程と時間を格納する構造体
28 struct TONE {
29     uint16_t frequency; // 周波数 (Hz), 0は無音
30     uint16_t duration;  // 持続時間 (ms)
31 };
32
33 // タイマー設定値を事前計算した構造体
34 struct TIMER_DATA {
35     uint16_t tgra_sound; // MTU21のTGRA値 (音声用)
36     uint16_t tgra_wait;  // MTU23のTGRA値 (待機時間用)
37     uint8_t is_sound;     // 0:無音, 1:音あり
38 };
39
40 // MIDIノート番号を周波数に変換するテーブル (A4=440Hz基準)
41 static const uint16_t midi_freq_table[128] = {
```

```

42     // C-1からC9まで
43     8, 9, 9, 10, 10, 11, 12, 12, 13, 14, 15, 15,
44     16, 17, 18, 19, 21, 22, 23, 24, 26, 28, 29, 31,
45     33, 35, 37, 39, 41, 44, 46, 49, 52, 55, 58, 62,
46     65, 69, 73, 78, 82, 87, 92, 98, 104, 110, 117, 123,
47     131, 139, 147, 156, 165, 175, 185, 196, 208, 220, 233, 247,
48     262, 277, 294, 311, 330, 349, 370, 392, 415, 440, 466, 494,
49     523, 554, 587, 622, 659, 698, 740, 784, 831, 880, 932, 988,
50     1047, 1109, 1175, 1245, 1319, 1397, 1480, 1568, 1661, 1760, 1865, 1976,
51     2093, 2217, 2349, 2489, 2637, 2794, 2960, 3136, 3322, 3520, 3729, 3951,
52     4186, 4435, 4699, 4978, 5274, 5588, 5920, 6272, 6645, 7040, 7459, 7902,
53     8372, 8870, 9397, 9956, 10548, 11175, 11840, 12544
54 };
55 //TFT定義
56 #define TFTDATA (*(volatile unsigned short *)0x08000000)
57 #define TFTCTRL (*(volatile unsigned short *)0x08000002)
58 #define _COL_WHITE (0xFFFF)
59 #define _COL_BLACK (0x0000)
60 //画面描画用
61 uint16_t framebuf[240 * 320];
62 //フォント読み込み
63 static const uint8_t Font6x8[][6] = {
64 #include "font6x8.inc"    // フォント別ファイルを後述
65 };
66
67 //GPIO定義
68 #define SW6 (PD.DR.BIT.B18)
69 #define SW5 (PD.DR.BIT.B17)
70 #define SW4 (PD.DR.BIT.B16)
71 #define LED6 (PE.DR.BIT.B11)
72 #define LED5 (PE.DR.BIT.B9)
73 #define LED_ON (0)
74 #define LED_OFF (1)
75
76 //標準入出力
77 #define printf ((int (*)(const char *, ...))0x00007c7c)
78 #define scanf ((int (*)(const char *, ...))0x00007cb8)
79
80 //SDカード
81 #define SD_CD (PE.DR.BIT.B11)
82 #define SD_CS (PE.DR.BIT.B9)
83 //SPK
84 #define SPK (PE.DR.BIT.B0)
85
86

```

```

87
88 // -----
89 struct disk_t
90 {
91     unsigned int First_sect_LBA;
92     unsigned int BPB_BytesPerSec;
93     unsigned int BPB_SecPerClus;
94     unsigned int BPB_RsvdSecCnt;
95     unsigned int BPB_NumFATs;
96     unsigned int BPB_RootEntCnt;
97     unsigned int BPB_FATSz16;
98     unsigned int First_RDE_sect;
99     unsigned int First_FAT_sect;
100    unsigned int First_Data_sect;
101 } DISK;
102
103 struct file_t
104 {
105     unsigned char Filename[12];
106     unsigned int n;           // RDE の(ゼロから数えて)  $n$  番目のファイル
107     unsigned int attr;       // 属性
108     unsigned int File_year;  // 年
109     unsigned int File_month; // 月
110     unsigned int File_date;  // 日
111     unsigned int File_hour;  // 時
112     unsigned int File_min;   // 分
113     unsigned int File_sec;   // 秒
114     unsigned int FstClusL0;
115     unsigned int FileSize;
116     uint16_t *Data; // ファイル本体
117 };
118 //TFT関連プロトタイプ宣言
119 void TFT_draw_char(int x, int y, char c, uint16_t color);
120 void TFT_draw_string(int x, int y, char *str, uint16_t color);
121 void show_file_info(struct file_t *file);
122 void show_all_files();
123 int find_and_load_file(struct file_t *target_file, char *target_filename);
124 int find_and_load_midi_file(struct file_t *target_file, char *target_filename);
125 int load_files_by_name();
126
127 uint16_t hue999_to_rgb565(uint16_t h);
128
129 void spawn_enemy_bullet(int enemy_x, int enemy_y);
130 //LCD関連
131 void LCD_inst(_SBYTE);

```



```

132 void LCD_data(_SBYTE);
133 void LCD_cursor(_UINT, _UINT);
134 void LCD_putch(_SBYTE);
135 void LCD_putstr(_SBYTE *);
136 void LCD_init(void);
137
138 // ボス関連の関数プロトタイプ
139 void init_boss();
140 void spawn_boss();
141 void update_boss();
142 void draw_boss();
143 void check_skill_boss_collision();
144 void boss_shoot_bullets();
145
146 // ウルト関連の関数プロトタイプ
147 void update_ult_gauge();
148 void activate_ult();
149 void update_ult();
150 void draw_ult_gauge();
151 void check_ult_input();
152 void init_beam();
153 void draw_beam();
154 void check_beam_collision();
155
156 // MIDI関連の関数プロトタイプ
157 int parse_midi_to_tone(struct file_t *midi_file, struct TONE *tone_array);
158 uint32_t read_variable_length(uint8_t *data, int *offset);
159 uint16_t midi_note_to_freq(uint8_t note);
160 int process_midi_track(uint8_t *track_data, int track_length, struct TONE *tone_array
    );
161 void get_File_as_bytes(struct file_t *file);
162 void convert_tone_to_timer_data(struct TONE *tone_array, int count, struct TIMER_DATA
    *timer_data);
163
164
165
166
167 //DMAC関連プロトタイプ宣言
168 void DMA0(void *SAR, void *DAR, uint8_t DM, uint8_t SM, int count);
169 void DMA1(void *SAR, void *DAR, uint8_t DM, uint8_t SM, int count);
170 void DMA2(void *SAR, void *DAR, uint8_t DM, uint8_t SM, int count);
171 void DMA3(void *SAR, void *DAR, uint8_t DM, uint8_t SM, int count);
172
173 // -----
174 // -- グローバル変数 --

```

```

175 unsigned char dt[512]; // セクタリード用ワーク
176 unsigned char fat[2048]; // FAT
177 unsigned char rde[512]; // RDE
178
179 struct file_t mari; // ファイル
180 struct file_t buttle; // ファイル
181 struct file_t boss_img; // ボス画像ファイル
182 struct file_t title_img; // タイトル画像ファイル
183 // UNオーエンは彼女なのか
184 struct file_t un_file;
185 struct TONE un[MAX_MIDI_EVENTS];
186 struct TIMER_DATA un_data[MAX_MIDI_EVENTS]; // 事前計算済みタイマーデータ
187 uint16_t midiData[2048]; // MIDIファイル用 (4KBまで対応)
188 // タイトル用music
189 struct file_t title_music_file;
190 struct TONE title_music[MAX_MIDI_EVENTS];
191 struct TIMER_DATA title_music_data[MAX_MIDI_EVENTS]; // 事前計算済みタイマーデータ
192 uint16_t titlemusicData[2048]; // MIDIファイル用 (4KBまで対応)
193 uint16_t FileData0[1024*4]; // ファイルデータ本体
194 // uint16_t FileData1[320*180 + 4]; // ファイルデータ本体
195 uint16_t FileData3[320*240 + 4]; // ファイルデータ本体
196 uint16_t FileData2[64*64 + 4]; // ボス画像データ本体 (64x64想定)
197 //DMAC管理用フラグ(これによって高速化を実現している。)
198 uint8_t DMAC0f = 0;
199 uint8_t DMAC1f = 0;
200 uint8_t DMAC2f = 0;
201 uint8_t DMAC3f = 0;
202 //音楽再生管理用
203 uint8_t music_playing = 0;
204
205 // MIDI音楽データを格納する配列
206 struct TONE music_data[MAX_MIDI_EVENTS];
207
208 // 障害物システム
209 #define MAX_OBSTACLES 8
210 struct obstacle_t {
211     int x;
212     int y;
213     int width;
214     int height;
215     uint16_t color;
216     int active;
217     int velocity;
218     int hp;
219     int max_hp;

```

```

220 } obstacles[MAX_OBSTACLES];
221
222 int obstacle_spawn_timer = 0;
223 int obstacle_spawn_interval = 30; // フレーム数
224 int collision_count = 0; // 当たり判定カウント用変数
225
226 // ボスシステム
227 struct boss_t {
228     int x;
229     int y;
230     int width;
231     int height;
232     uint16_t color;
233     int active;
234     int hp;
235     int max_hp;
236     int direction; // 移動方向 (1:下, -1:上)
237     int velocity;
238     int shoot_timer;
239     int shoot_pattern;
240     int invincible_timer; // 無敵時間
241 } boss;
242
243 int boss_spawn_timer = 0;
244 int boss_active = 0;
245
246 // スキルシステム (プレイヤーの弾)
247 #define MAX_SKILLS 10
248 struct skill_t {
249     int x;
250     int y;
251     int width;
252     int height;
253     uint16_t color;
254     int active;
255     int velocity;
256 } skills[MAX_SKILLS];
257
258 // 敵の弾幕システム
259 #define MAX_ENEMY_BULLETS 20
260 struct enemy_bullet_t {
261     int x;
262     int y;
263     int width;
264     int height;

```

```

265     uint16_t color;
266     int active;
267     int velocity_x;
268     int velocity_y;
269 } enemy_bullets[MAX_ENEMY_BULLETS];
270
271 // ゲームシステム
272 int score = 0;
273 int game_level = 1;
274 int picx = 0;
275 int picy = 0;
276
277 // ウルトシステム
278 int ult_gauge = 0;           // ウルトゲージ (0-100)
279 int ult_max_gauge = 100;    // ゲージ最大値
280 int ult_active = 0;         // ウルト発動中フラグ
281 int ult_timer = 0;          // ウルト継続時間 (フレーム数)
282 int ult_max_timer = 180;    // 3秒 = 60fps * 3
283 int ult_damage_total = 0;   // ウルト中の総ダメージ
284 int ult_available = 0;      // ウルト使用可能フラグ
285
286 // ビームシステム (ウルト攻撃用)
287 struct beam_t {
288     int active;
289     int x, y;
290     int width, height;
291     uint16_t color;
292     int damage_per_frame;
293 } ult_beam;
294
295 // フィールド関連の定数とデータ構造
296 #define MAX_FIELDS 3
297 #define FIELD_FOREST 0
298 #define FIELD_DESERT 1
299 #define FIELD_SPACE 2
300
301 // フィールド情報構造体
302 struct FieldInfo {
303     char name[16];
304     char bg_file[16]; // 背景ファイル名
305     int enemy_speed;   // 敵の移動速度倍率 (%)
306     int enemy_spawn_rate; // 敵の出現率倍率 (%)
307     int boss_hp_multiplier; // ボスHP倍率 (%)
308 };
309

```

```

310 // フィールド情報テーブル
311 struct FieldInfo field_data[MAX_FIELDS] = {
312     {"Forest Field", "FOREST .IMG", 80, 100, 100}, // 森フィールド：敵が少し遅い
313     {"Desert Field", "DESERT .IMG", 100, 120, 120}, // 砂漠フィールド：標準
314     {"Space Field", "SPACE .IMG", 120, 150, 150} // 宇宙フィールド：敵が速く多い
315 };
316
317 // 現在のフィールド
318 int current_field = FIELD_FOREST;
319
320 // -----
321 // -----
322 // 割り込み関数用フラグ群
323 int make1s = 0;
324 int count = 0;
325 int lastcount = 0;
326 int frag = 0;
327 int timing_frag = 0;
328 int tone_count;
329 int tone_count2;
330 int timing = 0;
331 #pragma interrupt INT_CMT1_CMIO
332 void INT_CMT1_CMIO() //1秒周期でfps(frame per second,画面更新頻度)更新, frag更新機能
    ,led点滅機能(1 / 2 = 0.5Hz周期)
333 {
334     CMT1.CMCSR.BIT.CMF = 0; // 割り込み要求をクリア
335
336     lastcount = count; //fpsを保持
337     frag = 1; //main関数で扱うようフラグ
338     count = 0; //fpsリセット
339     LED5 ^= 1;
340 }
341 #pragma interrupt INT_MTU2_CMI1
342 void INT_MTU2_CMI1() //スピーカー制御用関数. このタイマーのコンペアマッチを変更させる
    ことで任意の周波数を作り出せる.
343 {
344     MTU21.TSR.BIT.TGFA = 0; // 割り込み要求をクリア
345     // printf(".");
346
347     SPK ^= 1;
348 }
349 #pragma interrupt INT_MTU23_CMI2
350 void INT_MTU23_CMI2() //音楽制御用割り込み. 一つの音を生成するたびに, すぴか一用の割
    り込みとこの割り込みのコンペアマッチを変更させる. これにより, midiに対応した制御が
    楽になる.

```

```

351 {
352     MTU23.TSR.BIT.TGFA = 0;    // 割り込み要求をクリア
353     static uint16_t freq = 0,dur = 0,music4count=0,is_sound =0;
354     make1s++;
355     if(make1s >= 10){
356         make1s = 0;
357         timing++;
358         MTU2.TSTR.BIT.CST1 = 0; // タイマー停止
359         MTU2.TSTR.BIT.CST3 = 0; // タイマー停止
360         if(music_playing == 0){ //曲選択
361             freq = title_music_data[timing].tgra_sound;
362             dur = title_music_data[timing].tgra_wait;
363             music4count = tone_count2;
364             is_sound = title_music_data[timing].is_sound;
365         }else{
366             freq = un_data[timing].tgra_sound;
367             dur = un_data[timing].tgra_wait;
368             music4count = tone_count;
369             is_sound = un_data[timing].is_sound;
370         }
371         // printf("playing freq:%d,wait:%d\n",freq,dur);
372         if (timing < music4count && is_sound) { //is_soundで無音を制御
373             // 音を再生する - 事前計算済みの値を使用
374             MTU21.TGRA = freq;
375             MTU21.TCNT = 0; // カウンタクリア
376
377             // 持続時間 - 事前計算済みの値を使用
378             MTU23.TGRA = dur;
379             MTU23.TCNT = 0; // カウンタクリア
380             MTU2.TSTR.BIT.CST1 = 1; // タイマー開始
381             MTU2.TSTR.BIT.CST3 = 1; // タイマー開始
382         } else if (timing < music4count) { //無音時間.
383             // 無音期間 - 事前計算済みの値を使用
384             MTU23.TGRA = dur;
385             MTU23.TCNT = 0; // カウンタクリア
386             MTU2.TSTR.BIT.CST3 = 1; // タイマー開始
387         }else{
388             timing = 0;
389             MTU23.TGRA = 1;
390             MTU2.TSTR.BIT.CST3 = 1; // タイマー開始
391         }
392     }
393 }
394 #pragma interrupt INT_MTU2_CMI3

```

```

395 void INT_MTU2_CMI3() //7seg制御用割り込み．今スピーカーで再生している周波数を出力させ
    る．(ダイナミック点灯)
396 {
397     MTU24.TSR.BIT.TGFA = 0;    // 割り込み要求をクリア
398     static int num =0;
399     static int val = 0;
400     if(music_playing == 0){
401         val = title_music[timing].frequency;
402     }else{
403         val = un[timing].frequency;
404     }
405     num++;
406     if(num >= 3){
407         num = 0;
408     }
409     switch(num){
410         case 0:
411             val = val % 10;
412             break;
413         case 1:
414             val = (val / 10) % 10;
415             break;
416         case 2:
417             val = (val / 100) % 10;
418             break;
419     }
420     // printf("val:%d,keta:%d\n",val,keta);
421     PA.DR.BYTE.HL &= 0xF0;
422     if (num == 0)
423     {
424         DIG1 = 1;
425         DIG2 = 0;
426         DIG3 = 0;
427     }
428     else if (num == 1)
429     {
430         DIG1 = 0;
431         DIG2 = 1;
432         DIG3 = 0;
433     }
434     else
435     {
436         DIG1 = 0;
437         DIG2 = 0;
438         DIG3 = 1;

```

```

439     }
440     PA.DR.BYTE.HL |= val;
441 }
442 // int freq = 0, dur = 0;
443 // 割り込み, タイマー関連の初期化関数群
444 void init_CMT0()
445 {
446     STB.CR4.BIT._CMT = 0;
447
448     CMT0.CMCNT = 0;
449     CMT0.CMCSR.BIT.CKS = 1; // CKS設定 (0:1/8, 1:1/32, 2:1/128, 3:1/512)
450 }
451 void init_MTU2(){
452
453     STB.CR4.BIT._MTU2 = 0;
454     STB.CR4.BIT._ADO = 0;
455     // MTU2 ch0
456     MTU20.TCR.BIT.TPSC = 3; // 1/64選択
457     MTU20.TCR.BIT.CCLR = 1; // TGRAのコンペアマッチでクリア
458     MTU20.TGRA = 31250 - 1; // 100ms
459     MTU20.TIER.BIT.TTGE = 1; // A/D変換開始要求を許可
460     INTC.IPRD.BIT._MTU20G = 9; // 割り込み優先レベル = 11
461     // ADO
462     ADO.ADCSR.BIT.ADM = 3; // cycleモード
463     ADO.ADCSR.BIT.CH = 1; // AN0, An1
464     ADO.ADCSR.BIT.TRGE = 1; // MTU2からのトリガ有効
465     ADO.ADTSR.BIT.TRGOS = 1; // TGRAコンペアマッチでトリガ
466
467     //MTU2 ch1
468     MTU21.TCR.BIT.TPSC = 3; // 1/64選択
469     MTU21.TCR.BIT.CCLR = 1; // TGRAのコンペアマッチでクリア
470     MTU21.TGRA = 3125/20 - 1; // 100ms
471     MTU21.TIER.BIT.TGIEA = 1; // TGFAビットによる割り込み要求を許可
472     INTC.IPRD.BIT._MTU21G = 11; // 割り込み優先レベル = 10
473     //MTU2 ch3
474     MTU23.TCR.BIT.TPSC = 3; // 1/64選択
475     MTU23.TCR.BIT.CCLR = 1; // TGRAのコンペアマッチでクリア
476     MTU23.TGRA = 7812 - 1; // 100ms
477     MTU23.TIER.BIT.TGIEA = 1; // TGFAビットによる割り込み要求を許可
478     INTC.IPRE.BIT._MTU23G = 10; // 割り込み優先レベル = 9
479     //MTU2 ch4
480     MTU24.TCR.BIT.TPSC = 3; // 1/64選択
481     MTU24.TCR.BIT.CCLR = 1; // TGRAのコンペアマッチでクリア
482     MTU24.TGRA = 900 - 1; // 100ms
483     MTU24.TIER.BIT.TGIEA = 1; // TGFAビットによる割り込み要求を許可

```



```

484     INTC.IPRF.BIT._MTU24G = 9;  // 割り込み優先レベル = 9
485 }
486 void init_CMT1(){
487     STB.CR4.BIT._CMT = 0;        // CMTモジュールスタンバイの解除
488     CMT1.CMCSR.BIT.CKS = 3;      // 1/512
489     CMT1.CMCOR = 39062 - 1;      // 500ms
490     CMT1.CMCSR.BIT.CMIE = 1;     // コンペアマッチ割り込み許可
491     INTC.IPRJ.BIT._CMT1 = 8;     // 割り込み優先レベル = 8
492
493     set_imask(7);                // マスクビット = 7
494 }
495
496 // -----
497 void wait_us(unsigned int us)
498 {
499     unsigned int val;
500
501     val = us * 10 / 16;
502     if (val >= 0xffff)
503         val = 0xffff;
504
505     CMT0.CMCOR = val;
506     CMT0.CMCSR.BIT.CMF &= 0;
507     CMT.CMSTR.BIT.STRO = 1;
508
509     while (CMT0.CMCSR.BIT.CMF == 0)
510         ;
511     CMT0.CMCSR.BIT.CMF = 0;
512     CMT.CMSTR.BIT.STRO = 0;
513 }
514
515 // -----
516 void init_SCI2()
517 {
518     STB.CR3.BIT._SCI2 = 0;
519     SCI2.SCSCR.BYTE = 0x00;
520     SCI2.SCSMR.BIT.CA = 1;      // クロック同期式
521     SCI2.SCBRR = 12;           // 384kbps
522     SCI2.SCSDCR.BIT.DIR = 1;    // MSB first
523     wait_us(1);
524     PFC.PECRL3.BIT.PE10MD = 2;  // PE10 .. TxD
525     PFC.PECRL3.BIT.PE8MD = 2;   // PE8 .. SCK
526     PFC.PECRL2.BIT.PE7MD = 2;   // PE7 .. RxD
527     PFC.PEIORL.BIT.B12 = 0;     // PE12 .. WP入力
528     PFC.PEIORL.BIT.B11 = 0;     // PE11 .. CD入力

```

```

529     PFC.PEIORL.BIT.B9 = 1;      // PE9 .. CS出力
530     PE.DR.BIT.B9 = 1;          // CS初期値
531     SCI2.SCSCR.BYTE |= 0x30;    // TE, RE = 1
532 }
533
534 // -----
535 unsigned char SPI_tx_rx(unsigned char ch)
536 {
537     while (!SCI2.SCSSR.BIT.TDRE)
538         ;
539     SCI2.SCTDR = ch;
540     SCI2.SCSSR.BIT.TDRE = 0;
541
542     while (!SCI2.SCSSR.BIT.RDRF)
543         ;
544     ch = SCI2.SCRDR;
545     SCI2.SCSSR.BIT.RDRF = 0;
546     return (ch);
547 }
548
549 // -----
550 int card_exist()
551 {
552     return (PE.DR.BIT.B11 ? 0 : 1);
553 }
554
555 // -----
556 unsigned char calc_CRC7(unsigned char *data, int len)
557 {
558     int i, j;
559     char crc, dt;
560
561     crc = 0;
562     for (i = 0; i < len; i++)
563     {
564         dt = *data++;
565         for (j = 0; j < 8; j++)
566         {
567             crc <<= 1;
568             if ((crc & 0x80) ^ (dt & 0x80))
569                 crc ^= 0x09;
570             dt <<= 1;
571         }
572     }
573     return (crc & 0x7f);

```

```

574 }
575
576 // -----
577 unsigned char SD_send_cmd(unsigned char cmd, int arg)
578 {
579     int i;
580     unsigned char cmd_token[6], ret;
581
582     while (1)
583     {
584         if (SPI_tx_rx(0xff) == 0xff)
585             break;
586     }
587
588     cmd_token[0] = cmd;
589     cmd_token[1] = (arg >> 24) & 0xff;
590     cmd_token[2] = (arg >> 16) & 0xff;
591     cmd_token[3] = (arg >> 8) & 0xff;
592     cmd_token[4] = arg & 0xff;
593     cmd_token[5] = (calc_CRC7(cmd_token, 5) << 1) | 0x01;
594
595     for (i = 0; i < 6; i++)
596         SPI_tx_rx(cmd_token[i]);
597
598     do
599     {
600         ret = SPI_tx_rx(0xff);
601     } while (ret & 0x80);
602
603     return (ret);
604 }
605
606 // -----
607 int Enter_SPI_mode()
608 {
609     int i;
610     unsigned char ret;
611
612     SD_CS = 1; // CS negate
613
614     for (i = 0; i < 10; i++)
615         SPI_tx_rx(0xff);
616
617     SD_CS = 0; // CS assert
618

```

```

619     ret = SD_send_cmd(0x40, 0);
620     if (ret != 0x01)
621         return (-1);
622
623     do
624     {
625         ret = SD_send_cmd(0x41, 0);
626     } while (ret != 0x00);
627
628     return (1);
629 }
630
631 // -----
632
633 void read_sector(unsigned int sect, unsigned char *dt)
634 {
635     unsigned char ch;
636     int i;
637
638     ch = SD_send_cmd(0x51, sect * 512);
639     while (ch != 0xfe)
640         ch = SPI_tx_rx(0xff);
641
642     for (i = 0; i < 512; i++)
643         *dt++ = SPI_tx_rx(0xff);
644     SPI_tx_rx(0xff); // CRC分
645     SPI_tx_rx(0xff); // CRC分
646 }
647
648 // -----
649 void print_sector(unsigned int sect, unsigned char *dt)
650 {
651     int i, j;
652     unsigned int adr;
653
654     adr = sect * 512;
655
656     printf("          ");
657     for (i = 0; i < 16; i++)
658     {
659         printf("+%x ", i);
660         if (i == 7)
661             printf(" ");
662     }
663     printf("\n");

```

```

664
665     for (i = 0; i < 32; i++)
666     {
667         printf("%08x : ", adr);
668         for (j = 0; j < 16; j++)
669         {
670             printf("%02x ", dt[i * 16 + j]);
671             if (j == 7)
672                 printf(" ");
673         }
674         printf(" ");
675         for (j = 0; j < 16; j++)
676         {
677             if (isprint(dt[i * 16 + j]))
678                 printf("%c", dt[i * 16 + j]);
679             else
680                 printf(".");
681             if (j == 7)
682                 printf(" ");
683         }
684         printf("\n");
685         adr += 16;
686     }
687 }
688
689 // -----
690 // -----
691 // MBR を解析する
692 //
693 void ex_MBR()
694 {
695     // -- 演習 1 --
696     read_sector(0, dt);
697     // printf("dt[454]=0x%02x dt[455]=0x%02x dt[456]=0x%02x dt[457]=0x%02x\n", dt
698     // [454], dt[455], dt[456], dt[457]);
699     DISK.First_sect_LBA = dt[454] | (dt[455] << 8) | (dt[456] << 16) | (dt[457] <<
700     24);
701 }
702
703 // -----
704 // BPB を解析する
705 //
706 void ex_BPB()
707 {
708     // -- 演習 2 --

```

```

707     read_sector(DISK.First_sect_LBA, dt);
708     DISK.BPB_BytesPerSec = dt[11] | (dt[12] << 8);
709     DISK.BPB_SecPerClus = dt[13];
710     DISK.BPB_RsvdSecCnt = dt[14] | (dt[15] << 8);
711     DISK.BPB_NumFATs = dt[16];
712     DISK.BPB_RootEntCnt = dt[17] | (dt[18] << 8);
713     DISK.BPB_FATsSz16 = dt[22] | (dt[23] << 8);
714 }
715
716 // -----
717 // RDE を取得
718 //
719 void get_RDE(int n)
720 {
721     int i;
722     // -- 演習 3 --
723     DISK.First_RDE_sect = DISK.First_sect_LBA + DISK.BPB_RsvdSecCnt + (DISK.
        BPB_NumFATs * DISK.BPB_FATsSz16);
724     read_sector(DISK.First_RDE_sect + n, rde);
725     // print_sector(DISK.First_RDE_sect + n, rde);
726 }
727
728 // -----
729 // RDE の n 番目のファイル情報を取得
730 //
731 int get_file_info(struct file_t *file)
732 {
733     // -- 演習 4 --
734     int i;
735     unsigned int offset;
736
737     offset = file->n * 32;
738     if (rde[offset] == 0xe5 && rde[offset + 1] == 0x4a)
739     { // 削除ファイル
740         return (-1);
741     }
742     if (rde[offset] == 0x00)
743     { // 空のエントリ
744         file->Filename[0] = 0x00;
745         return (0);
746     }
747
748     // -- ファイル名 --
749     for (i = 0; i < 8; i++)
750         file->Filename[i] = rde[offset + i];

```

```

751
752     file->Filename[8] = '.';
753
754     // -- 拡張子 --
755     for (i = 8; i < 11; i++)
756         file->Filename[i + 1] = rde[offset + i];
757
758     // -- ここから編集 --
759     file->attr = rde[offset + 11];
760     file->File_hour = (rde[offset + 15] >> 3) & 0x1f;
761     file->File_min = ((rde[offset + 15] & 0x07) << 3) + (rde[offset + 14] >> 5);
762     file->File_sec = (rde[offset + 14] & 0x1f) * 2;
763     file->File_year = ((rde[offset + 17] >> 1) & 0x7f) + 1980;
764     file->File_month = ((rde[offset + 16] >> 5) & 0x07) + (((rde[offset + 17]) & 0x01
765         ) * 8);
766     file->File_date = rde[offset + 16] & 0x1f;
767     file->FstClusL0 = rde[offset + 26] | (rde[offset + 27] << 8);
768     file->FileSize = rde[offset + 28] | (rde[offset + 29] << 8) | (rde[offset + 30]
769         << 16) | (rde[offset + 31] << 24);
770
771     return (1);
772 }
773
774 // -----
775 // FAT を取得
776 // サイズが大きいファイルに対応するため、FAT は 2 セクタ分取得しておく
777 //
778 void get_FAT()
779 {
780     int i;
781     // -- 演習 5 --
782     DISK.First_FAT_sect = DISK.First_sect_LBA + DISK.BPB_RsvdSecCnt;
783     read_sector(DISK.First_FAT_sect, fat);
784     read_sector(DISK.First_FAT_sect + 1, &fat[512]);
785     read_sector(DISK.First_FAT_sect + 2, &fat[1024]);
786     read_sector(DISK.First_FAT_sect + 3, &fat[1536]);
787 }
788
789 // -----
790 // クラスタチェーンを表示
791 //
792 void ex_FAT(struct file_t *file)
793 {
794     // -- 演習 6 --
795     unsigned int cluster;
796

```

```

794     cluster = file->FstClusL0;
795     printf("%d ", cluster);
796     while (cluster != 0xffff)
797     {
798         cluster = fat[cluster * 2] | (fat[cluster * 2 + 1] << 8);
799         printf(" -> %d ", cluster);
800     }
801     printf("\n");
802 }
803
804 // -----
805 // ファイルを取得
806 //
807 void get_File(struct file_t *file)
808 {
809     // -- 演習 7 --
810     int i, j;
811     unsigned int sz;
812     unsigned int cluster;
813     unsigned int sz_16bit = 0; // 16bitデータのインデックス
814
815     DISK.First_Data_sect = DISK.First_sect_LBA + DISK.BPB_RsvdSecCnt + (DISK.
        BPB_NumFATs * DISK.BPB_FATSz16) + ((DISK.BPB_RootEntCnt * 32 + (DISK.
        BPB_BytesPerSec - 1)) / DISK.BPB_BytesPerSec);
816     cluster = file->FstClusL0;
817     // printf("First_Data_sect=0x%x\n", (DISK.First_Data_sect + (cluster - 2) * DISK.
        BPB_SecPerClus));
818     sz = 0;
819     while (cluster != 0xffff)
820     {
821         for (i = 0; i < DISK.BPB_SecPerClus; i++)
822         {
823             read_sector(DISK.First_Data_sect + (cluster - 2) * DISK.BPB_SecPerClus +
                i, dt);
824             // if(cluster == 252){
825
826             // printf("Reading sector: %u\n", DISK.First_Data_sect + (cluster - 2) *
                DISK.BPB_SecPerClus + i);
827             // print_sector(DISK.First_Data_sect + (cluster - 2) * DISK.
                BPB_SecPerClus + i, dt);
828             // }
829             for (j = 0; j < 512; j += 2) // 2バイトずつ処理
830             {
831                 if (sz + 1 < file->FileSize) {
832                     // 2つのuint8_tを1つのuint16_tに結合 (Little Endian)

```



```

833         file->Data[sz_16bit] = (dt[j+1] << 8) | dt[j];
834         sz_16bit++;
835         sz += 2;
836     } else if (sz < file->FileSize) {
837         // 奇数バイトの場合、下位バイトのみ設定
838         file->Data[sz_16bit] = dt[j];
839         sz_16bit++;
840         sz++;
841     }
842
843     if (sz >= file->FileSize)
844     {
845         printf("File read complete: %u bytes\n", sz);
846         break;
847     }
848 }
849 if (sz >= file->FileSize){
850     printf("File read complete: %u bytes\n", sz);
851     break;
852 }
853 }
854 if (sz >= file->FileSize)
855     break;
856 cluster = fat[cluster * 2] | (fat[cluster * 2 + 1] << 8);
857 // printf("Next cluster=%d\n", cluster);
858 }
859 }
860
861 // -----
862 // ファイルをバイト単位で取得 (MIDI等のバイナリファイル用)
863 //
864 void get_File_as_bytes(struct file_t *file)
865 {
866     int i, j;
867     unsigned int sz;
868     unsigned int cluster;
869
870     DISK.First_Data_sect = DISK.First_sect_LBA + DISK.BPB_RsvdSecCnt + (DISK.
        BPB_NumFATs * DISK.BPB_FATSz16) + ((DISK.BPB_RootEntCnt * 32 + (DISK.
        BPB_BytesPerSec - 1)) / DISK.BPB_BytesPerSec);
871     cluster = file->FstClusL0;
872     sz = 0;
873
874     uint8_t *byte_data = (uint8_t*)file->Data;
875

```

```

876     while (cluster != 0xffff)
877     {
878         for (i = 0; i < DISK.BPB_SecPerClus; i++)
879         {
880             read_sector(DISK.First_Data_sect + (cluster - 2) * DISK.BPB_SecPerClus +
                        i, dt);
881             for (j = 0; j < 512; j++)
882             {
883                 if (sz < file->FileSize) {
884                     byte_data[sz] = dt[j];
885                     sz++;
886                 } else {
887                     break;
888                 }
889             }
890             if (sz >= file->FileSize)
891                 break;
892         }
893         if (sz >= file->FileSize)
894             break;
895         cluster = fat[cluster * 2] | (fat[cluster * 2 + 1] << 8);
896     }
897 }
898
899 // -----
900 // ファイル内容を表示
901 //
902 void print_File(struct file_t *file)
903 {
904     int i;
905
906     for (i = 0; i < file->FileSize; i++)
907         printf("%c", file->Data[i]);
908     printf("\n");
909 }
910
911 void TFT_On(void)
912 {
913     TFTCTRL = 0x4000;
914 }
915 void init_CS2(void)
916 {
917     BSC.CS2BCR.LONG = 0x12490400;
918     BSC.CS2WCR = 0x000302C0;
919     PFC.PACRL4.BIT.PA15MD = 1;

```

```

920     PFC.PACRL2.BIT.PA6MD = 2;
921 }
922 void TFT_draw_pic(struct file_t *file,uint16_t x,uint8_t y)
923 {
924     // .IMGファイルの場合、3バイト目が0なので簡単に取得可能
925     uint8_t *raw_data = (uint8_t*)file->Data;
926     int w = raw_data[0] + (raw_data[1] << 8); // width (1バイト目)
927     int h = raw_data[3]; // height (2バイト目)
928     // raw_data[2] = 0 (3バイト目は0固定)
929     // for(int i = 0; i < 16; i++){
930     //     printf("raw_data[%d]=0x%02x\n", i, raw_data[i]);
931     // }
932
933     uint16_t *data_ptr = file->Data + 1; // 画像データの開始位置 (4バイト目から)
934
935     // 画像が画面外に完全にある場合は何もしない
936     if (x >= 320 || y >= 240) return;
937
938     // 描画範囲を画面内に制限
939     int draw_h = h;
940     int draw_w = w;
941
942     if (y + h > 240) draw_h = 240 - y;
943     if (x + w > 320) draw_w = 320 - x;
944     // printf("Drawing image at (%d, %d) with size %dx%d\n", x, y, draw_w, draw_h);
945
946     for (int i = 0; i < draw_h; i++)
947     {
948         for (int j = 0; j < draw_w; j++)
949         {
950             // フレームバッファの正しい位置を計算
951             int pos = (y + i) * 320 + (x + j);
952             if(*data_ptr != 0xFFFF) {
953                 // 透明色（白）なら描画しない
954                 framebuf[pos] = *data_ptr;
955             }
956             data_ptr++; // 次のピクセルデータへ
957         }
958
959         // 画面外にはみ出した部分のデータをスキップ
960         if (draw_w < w) {
961             data_ptr += (w - draw_w);
962         }
963     }
964 }

```

```

965
966
967
968
969
970 void TFT_draw_char(int x, int y, char ch, uint16_t color)
971 {
972     int i, j;
973     uint8_t line;
974     unsigned char font_index;
975
976     // ASCII 32(スペース)から127まで対応
977     // フォント配列は32から始まるので、32を引く
978     if (ch < 32 || ch > 127) {
979         font_index = 0; // スペース文字を使用
980     } else {
981         font_index = ch - 32; // ASCII 32からの相対位置
982     }
983
984     // フォントデータは6列×8行で格納されている
985     for (j = 0; j < 6; j++) // 6列をループ
986     {
987         line = Font6x8[font_index][j]; // j列目のデータ取得
988         for (i = 0; i < 8; i++) // 8行をループ
989         {
990             if (line & (1 << i)) // i行目のビットをチェック
991             {
992                 framebuffer[(y + i) * 320 + (x + j)] = color;
993             }
994             else
995             {
996                 framebuffer[(y + i) * 320 + (x + j)] = _COL_WHITE;
997             }
998         }
999     }
1000 }
1001 void TFT_draw_string(int x, int y, char *str, uint16_t color)
1002 {
1003     while (*str)
1004     {
1005         TFT_draw_char(x, y, *str++, color);
1006         x += 6;
1007     }
1008 }
1009 // show all files information

```

```

1010 void show_all_files()
1011 {
1012     struct file_t temp_file;
1013     temp_file.n = 3;  // n=0,1,2は予約領域なのでn=3から開始
1014     int i = 0;
1015     int t = 1;
1016     int status;
1017     printf("-----\n");
1018     while (status = get_file_info(&temp_file))
1019     {
1020         printf("s=%X ", i);
1021         printf("n=%d ", temp_file.n);
1022         if (status == -1)
1023         {
1024             printf("削除ファイル");
1025             // sprintf使用を避ける
1026             // TFT_draw_string(0,t*10, "DEL FILE", _COL_BLACK);
1027             t++;
1028             temp_file.n++;
1029         }
1030         else
1031         {
1032             for (int j = 0; j < 12; j++)
1033                 printf("%c", temp_file.Filename[j]);
1034             printf(" / ");
1035             // printf("属性 = 0x%x\n", temp_file.attr);
1036             printf("Time = %02d:%02d:%02d / ", temp_file.File_hour, temp_file.
                File_min, temp_file.File_sec);
1037             printf("Date = %4d/%02d/%02d", temp_file.File_year, temp_file.File_month,
                temp_file.File_date);
1038
1039             // sprintf使用を避けて、簡単な文字列表示
1040             // TFT_draw_string(16, t*10, temp_file.Filename, _COL_BLACK);
1041             // TFT_draw_char(0,t*10,(char)t + '0',_COL_BLACK);
1042             // printf("FileSize = %d\n", temp_file.FileSize);
1043             // printf("FstClusLO = %d\n", temp_file.FstClusLO);
1044             temp_file.n++;
1045             t++;
1046         }
1047
1048         if (temp_file.n == 9 && i == 1)
1049         {
1050             temp_file.n++;
1051         }
1052         if (temp_file.n > (DISK.BPB_RootEntCnt / 32) - 1)

```

```

1053     {
1054         i++;
1055         get_RDE(i);
1056         temp_file.n = 0;
1057     }
1058     printf("\n");
1059     // ex_FAT(&temp_file);
1060 }
1061 // TFT_draw_string(160,230,"show", _COL_BLACK);
1062 }
1063
1064 // show specific file information by filename
1065 void show_file_info(struct file_t *file)
1066 {
1067     printf("File Information:\n");
1068     printf("Filename: ");
1069     for (int j = 0; j < 12; j++)
1070         printf("%c", file->Filename[j]);
1071     printf("\n");
1072     printf("Attribute: 0x%x\n", file->attr);
1073     printf("Time: %02d:%02d:%02d\n", file->File_hour, file->File_min, file->File_sec
        );
1074     printf("Date: %4d/%02d/%02d\n", file->File_year, file->File_month, file->
        File_date);
1075     printf("FileSize: %d bytes\n", file->FileSize);
1076     printf("First Cluster: %d\n", file->FstClusL0);
1077     printf("-----\n");
1078 }
1079 // -----
1080 // 簡易ファイルシステム
1081 // -----
1082 // ファイル名を比較する関数（大文字小文字を考慮）
1083 int strcmp_filename(char *str1, char *str2) {
1084     int i = 0;
1085     while (str1[i] != '\0' && str2[i] != '\0') {
1086         if (str1[i] != str2[i]) {
1087             return 0; // 不一致
1088         }
1089         i++;
1090     }
1091     return (str1[i] == '\0' && str2[i] == '\0') ? 1 : 0; // 一致の場合は1
1092 }
1093
1094 // 指定したファイル名のファイルを見つけて読み込む関数
1095 int find_and_load_file(struct file_t *target_file, char *target_filename) {

```

```

1096     struct file_t temp_file;
1097     temp_file.n = 3; // n=0,1,2は予約領域なのでn=3から開始
1098     int status;
1099     int sector_num = 0;
1100
1101     printf("Searching for file: %s\n", target_filename);
1102
1103     get_RDE(sector_num);
1104
1105     while (1) {
1106         status = get_file_info(&temp_file);
1107         // printf("Status: %d, File number: %d\n", status, temp_file.n);
1108         if (status == 0) {
1109             // 空きエンTRIESに到達
1110             if (temp_file.n > (DISK.BPB_RootEntCnt / 32) - 1) {
1111                 sector_num++;
1112                 get_RDE(sector_num);
1113                 temp_file.n = 0;
1114                 continue;
1115             } else {
1116                 break; // ファイルが見つからない
1117             }
1118         }
1119
1120         if (status == 1) {
1121             // 有効なファイルエンTRIES
1122
1123             // ファイル名が一致するかチェック
1124             if (strcmp_filename(temp_file.Filename, target_filename)) {
1125                 // ファイルが見つかった場合、target_fileにコピー
1126                 // ファイル情報をコピー (Dataポインタは保持)
1127                 uint16_t *saved_data = target_file->Data;
1128                 *target_file = temp_file;
1129                 target_file->Data = saved_data; // Dataポインタを復元
1130
1131                 // ファイル内容を読み込み
1132                 printf("File found: %s, Size: %d bytes\n", target_filename,
1133                     target_file->FileSize);
1134                 get_File(target_file);
1135
1136                 return 1; // 成功
1137             }
1138         }
1139
1140         temp_file.n++;

```

```

1140
1141     if (temp_file.n > (DISK.BPB_RootEntCnt / 32) - 1) {
1142         sector_num++;
1143         get_RDE(sector_num);
1144         temp_file.n = 0; // 新しいセクタでは0から開始
1145     }
1146 }
1147
1148 printf("File not found: %s\n", target_filename);
1149 return 0; // ファイルが見つからない
1150 }
1151
1152 // MIDIファイル用の特別な読み込み関数
1153 int find_and_load_midi_file(struct file_t *target_file, char *target_filename) {
1154     struct file_t temp_file;
1155     temp_file.n = 3; // n=0,1,2は予約領域なのでn=3から開始
1156     int status;
1157     int sector_num = 0;
1158
1159     printf("Searching for MIDI file: %s\n", target_filename);
1160
1161     get_RDE(sector_num);
1162
1163     while (1) {
1164         status = get_file_info(&temp_file);
1165
1166         if (status == 0) {
1167             // 空きエントリに到達
1168             if (temp_file.n > (DISK.BPB_RootEntCnt / 32) - 1) {
1169                 sector_num++;
1170                 get_RDE(sector_num);
1171                 temp_file.n = 0;
1172                 continue;
1173             } else {
1174                 break; // ファイルが見つからない
1175             }
1176         }
1177
1178         if (status == 1) {
1179             // 有効なファイルエントリ
1180
1181             // ファイル名が一致するかチェック
1182             if (strcmp_filename(temp_file.Filename, target_filename)) {
1183                 // ファイルが見つかった場合、target_fileにコピー
1184                 // ファイル情報をコピー (Dataポインタは保持)

```



```

1185         uint16_t *saved_data = target_file->Data;
1186         *target_file = temp_file;
1187         target_file->Data = saved_data; // Dataポインタを復元
1188
1189         // MIDIファイルはバイト単位で読み込み
1190         get_File_as_bytes(target_file);
1191
1192         return 1; // 成功
1193     }
1194 }
1195
1196 temp_file.n++;
1197
1198 if (temp_file.n > (DISK.BPB_RootEntCnt / 32) - 1) {
1199     sector_num++;
1200     get_RDE(sector_num);
1201     temp_file.n = 0; // 新しいセクタでは0から開始
1202 }
1203 }
1204
1205 printf("MIDI file not found: %s\n", target_filename);
1206 return 0; // ファイルが見つからない
1207 }
1208
1209 // 複数のファイルを名前で自動読み込み
1210 int load_files_by_name() {
1211     int success_count = 0;
1212
1213     printf("Auto-loading files by name...\n");
1214
1215     // MARIO.IMGを読み込み
1216     mari.Data = FileData0;
1217     if (find_and_load_file(&mari, "MARIO.IMG")) {
1218         printf("MARIO.IMG loaded\n");
1219         TFT_draw_string(0, 8, "MARIO.IMG loaded", _COL_BLACK);
1220         success_count++;
1221     } else {
1222         printf("MARIO.IMG not found\n");
1223         TFT_draw_string(0, 8, "MARIO.IMG not found", _COL_BLACK);
1224     }
1225     DMA0((void *)framebuf, (void*) 0x08000000, 0, 1, 320 * 240);
1226
1227
1228     // BOSS.IMGを読み込み
1229     boss_img.Data = FileData2;

```

```

1230     if (find_and_load_file(&boss_img, "BOSS    .IMG")) {
1231         printf("BOSS.IMG loaded\n");
1232         TFT_draw_string(0, 24, "BOSS.IMG loaded", _COL_BLACK);
1233         success_count++;
1234     } else {
1235         printf("BOSS.IMG not found\n");
1236         TFT_draw_string(0, 24, "BOSS.IMG not found", _COL_BLACK);
1237     }
1238     DMA0((void *)framebuf, (void*) 0x08000000, 0, 1, 320 * 240);
1239
1240     // MIDIファイルを読み込み（複数の名前を試行）
1241     un_file.Data = midiData;
1242     if (find_and_load_midi_file(&un_file, "UN    .MID")) {
1243         tone_count = parse_midi_to_tone(&un_file, un);
1244         // TONE配列からTIMER_DATA配列へ変換
1245         convert_tone_to_timer_data(un, tone_count, un_data);
1246         printf("MIDI file loaded\n");
1247         TFT_draw_string(0, 32, "MIDI file loaded", _COL_BLACK);
1248         success_count++;
1249     } else {
1250         printf("No MIDI file found\n");
1251         TFT_draw_string(0, 32, "No MIDI file found", _COL_BLACK);
1252     }
1253     DMA0((void *)framebuf, (void*) 0x08000000, 0, 1, 320 * 240);
1254     title_music_file.Data = titlemusicData;
1255     if( find_and_load_midi_file(&title_music_file, "TITLE .MID")) {
1256         tone_count2 = parse_midi_to_tone(&title_music_file, title_music);
1257         convert_tone_to_timer_data(title_music, tone_count2, title_music_data);
1258         // TONE配列からTIMER_DATA配列へ変換
1259         printf("TITLE.MID loaded\n");
1260         TFT_draw_string(0, 40, "TITLE.MID loaded", _COL_BLACK);
1261         success_count++;
1262     } else {
1263         printf("TITLE.MID not found\n");
1264         TFT_draw_string(0, 40, "TITLE.MID not found", _COL_BLACK);
1265     }
1266     DMA0((void *)framebuf, (void*) 0x08000000, 0, 1, 320 * 240);
1267
1268     title_img.Data = FileData3;
1269     if (find_and_load_file(&title_img, "TITLE    .IMG")) {
1270         printf("TITLE.IMG loaded\n");
1271         TFT_draw_string(0, 48, "TITLE.IMG loaded", _COL_BLACK);
1272         success_count++;
1273     } else {
1274         printf("TITLE.IMG not found\n");

```

```

1275     TFT_draw_string(0, 48, "TITLE.IMG not found", _COL_BLACK);
1276 }
1277 DMA0((void *)framebuf, (void*) 0x08000000, 0, 1, 320 * 240);
1278
1279 printf("Auto-loading completed: %d/5 files loaded\n", success_count);
1280
1281 return success_count;
1282 }
1283 // -----
1284 // 400段階で色相を一周する関数
1285 // -----
1286 uint16_t hue999_to_rgb565(uint16_t h)
1287 {
1288     // 0~999 → 0~359
1289     uint16_t hue = (h * 360) / 1000;
1290
1291     uint8_t r = 0, g = 0, b = 0;
1292
1293     uint16_t region = hue / 60;    // 0~5
1294     uint16_t f = hue % 60;        // 0~59
1295
1296     uint8_t p = 0;
1297     uint8_t q = 255 - (255 * f) / 60;
1298     uint8_t t = (255 * f) / 60;
1299
1300     switch (region) {
1301         case 0: r = 255; g = t; b = p; break; // 赤→黄
1302         case 1: r = q; g = 255; b = p; break; // 黄→緑
1303         case 2: r = p; g = 255; b = t; break; // 緑→水
1304         case 3: r = p; g = q; b = 255; break; // 水→青
1305         case 4: r = t; g = p; b = 255; break; // 青→紫
1306         default: r = 255; g = p; b = q; break; // 紫→赤
1307     }
1308
1309     // RGB888 → RGB565
1310     return ((r & 0xF8) << 8)
1311         | ((g & 0xFC) << 3)
1312         | (b >> 3);
1313 }
1314
1315 // 矩形描画関数 (fillrect風)
1316 void TFT_fill_rect(int x, int y, int width, int height, uint16_t color)
1317 {
1318     int i, j;
1319

```

```

1320 // 画面範囲チェック
1321 if (x < 0 || y < 0 || x >= 320 || y >= 180) return;
1322 if (x + width > 320) width = 320 - x;
1323 if (y + height > 180) height = 180 - y;
1324
1325 for (i = 0; i < height; i++)
1326 {
1327     for (j = 0; j < width; j++)
1328     {
1329         framebuf[(y + i) * 320 + (x + j)] = color;
1330     }
1331 }
1332 }
1333 // -----
1334 // 敵管理システム
1335 // -----
1336 // 障害物（敵）初期化
1337 void init_obstacles()
1338 {
1339     int i;
1340     for (i = 0; i < MAX_OBSTACLES; i++)
1341     {
1342         obstacles[i].active = 0;
1343         obstacles[i].x = 320;
1344         obstacles[i].y = 0;
1345         obstacles[i].width = 25;
1346         obstacles[i].height = 25;
1347         obstacles[i].velocity = 2;
1348         obstacles[i].color = 0xF800; // 赤色
1349         obstacles[i].hp = 3;
1350         obstacles[i].max_hp = 3;
1351     }
1352 }
1353
1354 // 新しい敵を生成
1355 void spawn_obstacle()
1356 {
1357     int i;
1358     for (i = 0; i < MAX_OBSTACLES; i++)
1359     {
1360         if (!obstacles[i].active)
1361         {
1362             obstacles[i].active = 1;
1363             obstacles[i].x = 320;
1364             obstacles[i].y = 10 + (count * 37) % (160); // y位置を制限

```

```

1365     obstacles[i].width = 20 + ((count * 17) % 15); // 幅20-34
1366     obstacles[i].height = 20 + ((count * 23) % 15); // 高さ20-34
1367
1368     // フィールドの速度倍率を適用
1369     int base_velocity = 1 + ((count * 13) % 3); // 基本速度1-3
1370     obstacles[i].velocity = (base_velocity * field_data[current_field].
        enemy_speed) / 100;
1371     if (obstacles[i].velocity < 1) obstacles[i].velocity = 1; // 最低速度保証
1372
1373     obstacles[i].max_hp = 2 + ((count * 11) % 4); // HP 2-5
1374     obstacles[i].hp = obstacles[i].max_hp;
1375     obstacles[i].color = hue999_to_rgb565((count * 150) % 1000); // カラフル
        な色
1376     break;
1377 }
1378 }
1379 }
1380
1381 // 敵更新 (弾幕発射機能付き)
1382 void update_obstacles()
1383 {
1384     int i;
1385
1386     // 既存の敵を移動
1387     for (i = 0; i < MAX_OBSTACLES; i++)
1388     {
1389         if (obstacles[i].active)
1390         {
1391             obstacles[i].x -= obstacles[i].velocity;
1392
1393             // 敵が弾を発射する (確率的に)
1394             if ((count + i * 7) % 10 == 0 && obstacles[i].x > 50 && obstacles[i].x <
                300)
1395             {
1396                 spawn_enemy_bullet(obstacles[i].x, obstacles[i].y + obstacles[i].
                    height/2);
1397                 // printf("spawned");
1398             }
1399
1400             // 画面外に出たら非アクティブに
1401             if (obstacles[i].x + obstacles[i].width < 0)
1402             {
1403                 obstacles[i].active = 0;
1404             }
1405         }

```

```

1406     }
1407
1408     // 新しい敵を生成するタイミング
1409     obstacle_spawn_timer++;
1410
1411     // フィールドのスポン率倍率を適用
1412     int adjusted_spawn_interval = (obstacle_spawn_interval * 100) / field_data[
        current_field].enemy_spawn_rate;
1413     if (adjusted_spawn_interval < 10) adjusted_spawn_interval = 10; // 最低間隔保証
1414
1415     if (obstacle_spawn_timer >= adjusted_spawn_interval)
1416     {
1417         spawn_obstacle();
1418         obstacle_spawn_timer = 0;
1419
1420         // 生成間隔を少しずつ変化させる
1421         obstacle_spawn_interval = 40 + (count % 60);
1422     }
1423 }
1424
1425 // 障害物描画 (HPバー付き)
1426 void draw_obstacles()
1427 {
1428     int i;
1429     for (i = 0; i < MAX_OBSTACLES; i++)
1430     {
1431         if (obstacles[i].active)
1432         {
1433             // HPに応じて色を変える
1434             uint16_t base_color = obstacles[i].color;
1435             if (obstacles[i].hp < obstacles[i].max_hp)
1436             {
1437                 // ダメージを受けた敵は少し暗くする
1438                 base_color = (base_color >> 1) & 0x7BEF; // 明度を下げる
1439             }
1440
1441             // 敵本体を描画
1442             TFT_fill_rect(obstacles[i].x, obstacles[i].y,
1443                 obstacles[i].width, obstacles[i].height,
1444                 base_color);
1445
1446             // HPバーを描画 (敵の上部)
1447             if (obstacles[i].max_hp > 1)
1448             {

```

```

1449         int hp_bar_width = (obstacles[i].width * obstacles[i].hp) / obstacles
           [i].max_hp;
1450
1451         // HPバー背景 (赤)
1452         TFT_fill_rect(obstacles[i].x, obstacles[i].y - 3,
1453                     obstacles[i].width, 2, 0xF800);
1454
1455         // HPバー (緑)
1456         if (hp_bar_width > 0)
1457         {
1458             TFT_fill_rect(obstacles[i].x, obstacles[i].y - 3,
1459                         hp_bar_width, 2, 0x07E0);
1460         }
1461     }
1462 }
1463 }
1464 }
1465
1466 // 当たり判定チェック関数
1467 int check_collision(int player_x, int player_y, int player_w, int player_h,
1468                   int obstacle_x, int obstacle_y, int obstacle_w, int obstacle_h)
1469 {
1470     // 矩形同士の当たり判定
1471     if (player_x < obstacle_x + obstacle_w &&
1472         player_x + player_w > obstacle_x &&
1473         player_y < obstacle_y + obstacle_h &&
1474         player_y + player_h > obstacle_y)
1475     {
1476         return 1; // 当たっている
1477     }
1478     return 0; // 当たっていない
1479 }
1480 // -----
1481 // プレイヤーシステム
1482 // -----
1483 // プレイヤーと敵の当たり判定処理
1484 void check_player_collision(int player_x, int player_y, int player_w, int player_h)
1485 {
1486     int i;
1487     for (i = 0; i < MAX_OBSTACLES; i++)
1488     {
1489         if (obstacles[i].active)
1490         {
1491             if (check_collision(player_x + player_w/4, player_y + player_h/4,

```

```

1492         player_w/2, player_h/2, // プレイヤーの当たり判定を小さ
           <
1493         obstacles[i].x, obstacles[i].y,
1494         obstacles[i].width, obstacles[i].height))
1495     {
1496         // 当たり判定が発生
1497         collision_count += 5; // 敵との接触は大ダメージ
1498         obstacles[i].hp--; // 敵のHPを1減らす
1499
1500         if (obstacles[i].hp <= 0)
1501         {
1502             obstacles[i].active = 0; // HPが0になったら敵を削除
1503             score += 5; // 敵を倒したボーナス
1504             ult_gauge += 5; // ウルトゲージを5増加
1505             if (ult_gauge >= ult_max_gauge) {
1506                 ult_gauge = ult_max_gauge;
1507                 ult_available = 1; // ウルト使用可能
1508             }
1509         }
1510
1511         break; // 一フレームで複数の当たり判定を防ぐ
1512     }
1513 }
1514 }
1515 }
1516
1517 // スキル初期化
1518 void init_skills()
1519 {
1520     int i;
1521     for (i = 0; i < MAX_SKILLS; i++)
1522     {
1523         skills[i].active = 0;
1524         skills[i].x = 0;
1525         skills[i].y = 0;
1526         skills[i].width = 8;
1527         skills[i].height = 4;
1528         skills[i].velocity = 8;
1529         skills[i].color = 0x07FF; // シアン色
1530     }
1531 }
1532
1533 // 新しいスキルを生成
1534 void spawn_skill(int player_x, int player_y, int player_w, int player_h)
1535 {

```



```

1536     int c = 0;
1537     int i;
1538     for (i = 0; i < MAX_SKILLS; i++)
1539     {
1540         if (!skills[i].active)
1541         {
1542             skills[i].active = 1;
1543             skills[i].x = player_x + player_w; // プレイヤーの右端から発射
1544             if(c == 0){
1545                 skills[i].y = player_y + player_h / 2; // プレイヤーの中央高さ
1546             }else if(c == 1){
1547                 skills[i].y = player_y + player_h / 2 - 10; // 少し上向き
1548             }else{
1549                 skills[i].y = player_y + player_h / 2 + 10; // 少し下向き
1550             }
1551             skills[i].width = 8;
1552             skills[i].height = 4;
1553             skills[i].velocity = 20;
1554             skills[i].color = 0x07FF; // シアン色
1555             c++;
1556         }
1557         if (c >= 3) {
1558             break; // 最大3発まで同時に発射
1559         }
1560     }
1561 }
1562
1563 // スキル更新
1564 void update_skills()
1565 {
1566     int i;
1567
1568     // 既存のスキルを移動
1569     for (i = 0; i < MAX_SKILLS; i++)
1570     {
1571         if (skills[i].active)
1572         {
1573             skills[i].x += skills[i].velocity;
1574
1575             // 画面外に出たら非アクティブに
1576             if (skills[i].x > 320)
1577             {
1578                 skills[i].active = 0;
1579             }
1580         }

```

```

1581     }
1582 }
1583
1584 // スキル描画
1585 void draw_skills()
1586 {
1587     int i;
1588     for (i = 0; i < MAX_SKILLS; i++)
1589     {
1590         if (skills[i].active)
1591         {
1592             TFT_fill_rect(skills[i].x, skills[i].y,
1593                 skills[i].width, skills[i].height,
1594                 skills[i].color);
1595         }
1596     }
1597 }
1598
1599 // スキルと障害物の当たり判定処理
1600 void check_skill_collision()
1601 {
1602     int i, j;
1603
1604     for (i = 0; i < MAX_SKILLS; i++)
1605     {
1606         if (skills[i].active)
1607         {
1608             for (j = 0; j < MAX_OBSTACLES; j++)
1609             {
1610                 if (obstacles[j].active)
1611                 {
1612                     if (check_collision(skills[i].x, skills[i].y, skills[i].width,
1613                         skills[i].height,
1614                         obstacles[j].x, obstacles[j].y,
1615                         obstacles[j].width, obstacles[j].height))
1616                     {
1617                         // スキルと敵が当たった
1618                         skills[i].active = 0; // スキルを削除
1619                         obstacles[j].hp--; // 敵のHPを1減らす
1620
1621                         // ウルト中のダメージカウント
1622                         if (ult_active) {
1623                             ult_damage_total++; // ウルト中のダメージをカウント
1624                         }

```

```

1625         if (obstacles[j].hp <= 0)
1626         {
1627             obstacles[j].active = 0; // HPが0になったら敵を削除
1628             score += 10; // 敵を倒したスコア
1629             ult_gauge += 10; // ウルトゲージを10増加
1630             if (ult_gauge >= ult_max_gauge) {
1631                 ult_gauge = ult_max_gauge;
1632                 ult_available = 1; // ウルト使用可能
1633             }
1634         }
1635     else
1636     {
1637         score += 2; // ダメージを与えたスコア
1638     }
1639
1640     break; // このスキルの処理を終了
1641 }
1642 }
1643 }
1644 }
1645 }
1646 }
1647
1648 // 敵の弾幕初期化
1649 void init_enemy_bullets()
1650 {
1651     int i;
1652     for (i = 0; i < MAX_ENEMY_BULLETS; i++)
1653     {
1654         enemy_bullets[i].active = 0;
1655         enemy_bullets[i].x = 0;
1656         enemy_bullets[i].y = 0;
1657         enemy_bullets[i].width = 4;
1658         enemy_bullets[i].height = 4;
1659         enemy_bullets[i].velocity_x = -4;
1660         enemy_bullets[i].velocity_y = 0;
1661         enemy_bullets[i].color = 0xFFE0; // 黄色
1662     }
1663 }
1664
1665 // 敵の弾を生成
1666 void spawn_enemy_bullet(int enemy_x, int enemy_y)
1667 {
1668     int i;
1669     for (i = 0; i < MAX_ENEMY_BULLETS; i++)

```

```

1670     {
1671         if (!enemy_bullets[i].active)
1672         {
1673             enemy_bullets[i].active = 1;
1674             enemy_bullets[i].x = enemy_x;
1675             enemy_bullets[i].y = enemy_y;
1676             enemy_bullets[i].width = 4;
1677             enemy_bullets[i].height = 4;
1678
1679             // プレイヤー方向に向かう弾の計算 (簡易版)
1680             int dx = picx - enemy_x;
1681             int dy = picy - enemy_y;
1682
1683             // 弾幕パターン：複数方向に発射
1684             if (i % 3 == 0) {
1685                 enemy_bullets[i].velocity_x = -3;
1686                 enemy_bullets[i].velocity_y = 0;
1687             } else if (i % 3 == 1) {
1688                 enemy_bullets[i].velocity_x = -3;
1689                 enemy_bullets[i].velocity_y = (dy > 0) ? 2 : -2;
1690             } else {
1691                 enemy_bullets[i].velocity_x = -3;
1692                 enemy_bullets[i].velocity_y = (dy > 0) ? -2 : 2;
1693             }
1694
1695             enemy_bullets[i].color = hue999_to_rgb565((count * 50) % 1000);
1696             break;
1697         }
1698     }
1699 }
1700
1701 // 敵の弾更新
1702 void update_enemy_bullets()
1703 {
1704     int i;
1705
1706     for (i = 0; i < MAX_ENEMY_BULLETS; i++)
1707     {
1708         if (enemy_bullets[i].active)
1709         {
1710             enemy_bullets[i].x += enemy_bullets[i].velocity_x;
1711             enemy_bullets[i].y += enemy_bullets[i].velocity_y;
1712
1713             // 画面外に出たら非アクティブに
1714             if (enemy_bullets[i].x < 0 || enemy_bullets[i].x > 320 ||

```

```

1715         enemy_bullets[i].y < 0 || enemy_bullets[i].y > 180)
1716     {
1717         enemy_bullets[i].active = 0;
1718     }
1719 }
1720 }
1721 }
1722
1723 // 敵の弾描画
1724 void draw_enemy_bullets()
1725 {
1726     int i;
1727     for (i = 0; i < MAX_ENEMY_BULLETS; i++)
1728     {
1729         if (enemy_bullets[i].active)
1730         {
1731             TFT_fill_rect(enemy_bullets[i].x, enemy_bullets[i].y,
1732                 enemy_bullets[i].width, enemy_bullets[i].height,
1733                 enemy_bullets[i].color);
1734         }
1735     }
1736 }
1737
1738 // プレイヤーと敵の弾の当たり判定
1739 void check_player_bullet_collision(int player_x, int player_y, int player_w, int
    player_h)
1740 {
1741     int i;
1742     for (i = 0; i < MAX_ENEMY_BULLETS; i++)
1743     {
1744         if (enemy_bullets[i].active)
1745         {
1746             if (check_collision(player_x + player_w/4, player_y + player_h/4,
1747                 player_w/2, player_h/2, // プレイヤーの当たり判定を小さ
                    く
1748                 enemy_bullets[i].x, enemy_bullets[i].y,
1749                 enemy_bullets[i].width, enemy_bullets[i].height))
1750             {
1751                 // プレイヤーと敵の弾が当たった
1752                 enemy_bullets[i].active = 0; // 弾を削除
1753                 collision_count++; // ダメージカウント
1754
1755                 // ダメージ演出（プレイヤーを一瞬白く）
1756                 break; // 一フレームで複数の当たり判定を防ぐ
1757             }

```

```

1758     }
1759 }
1760 }
1761
1762 // -----
1763 // ボス敵システム
1764 // -----
1765
1766 // ボス初期化
1767 void init_boss()
1768 {
1769     boss.active = 0;
1770     boss.x = 280;
1771     boss.y = 40;
1772     boss.width = 35;
1773     boss.height = 40;
1774     boss.color = 0xF81F; // マゼンタ
1775
1776     // フィールドのボスHP倍率を適用
1777     int base_hp = 1000;
1778     boss.hp = (base_hp * field_data[current_field].boss_hp_multiplier) / 100;
1779     boss.max_hp = boss.hp;
1780
1781     boss.direction = 1; // 初期は下向き
1782     boss.velocity = 1;
1783     boss.shoot_timer = 0;
1784     boss.shoot_pattern = 0;
1785     boss.invincible_timer = 0;
1786     boss_active = 0;
1787 }
1788
1789 // ボス生成
1790 void spawn_boss()
1791 {
1792     if (!boss.active && score > 200) { // スコア200以上でボス出現
1793         // MTU2.TSTR.BIT.CST3 = 1; // MTU2 CH3スタート
1794         boss.active = 1;
1795         boss.x = 280;
1796         boss.y = 40;
1797         boss.hp = boss.max_hp;
1798         boss.direction = 1;
1799         boss.shoot_timer = 0;
1800         boss.invincible_timer = 0;
1801         boss_active = 1;
1802     }

```

```

1803 }
1804
1805 // ボス更新
1806 void update_boss()
1807 {
1808     if (!boss.active) return;
1809
1810     // 無敵時間を減らす
1811     if (boss.invincible_timer > 0) {
1812         boss.invincible_timer--;
1813     }
1814
1815     // 上下移動
1816     boss.y += boss.direction * boss.velocity;
1817
1818     // 画面端で反転
1819     if (boss.y <= 0) {
1820         boss.y = 0;
1821         boss.direction = 1; // 下向きに
1822     }
1823     if (boss.y + boss.height >= 160) {
1824         boss.y = 160 - boss.height;
1825         boss.direction = -1; // 上向きに
1826     }
1827
1828     // 弾発射
1829     boss_shoot_bullets();
1830
1831     // HPが0になったら消去
1832     if (boss.hp <= 0) {
1833         boss.active = 0;
1834         boss_active = 0;
1835         score += 1000; // ボス撃破ボーナス
1836     }
1837 }
1838
1839 // ボス描画
1840 void draw_boss()
1841 {
1842     if (!boss.active) return;
1843
1844     // ボス画像を描画
1845     if (boss_img.Data != 0) {
1846         // 無敵時間中のエフェクト処理
1847         if (boss.invincible_timer > 0 && (boss.invincible_timer % 4 < 2)) {

```

```

1848 // 無敵時間中は画像の色調を変更（簡易的に白っぽくする）
1849 // 元の画像データを一時的に変更してから描画
1850 uint8_t *raw_data = (uint8_t*)boss_img.Data;
1851 int w = raw_data[0] + (raw_data[1] << 8); // width
1852 int h = raw_data[3]; // height
1853 uint16_t *data_ptr = boss_img.Data + 1; // 画像データの開始位置
1854
1855 // 簡易的な白色フラッシュ効果：一時的に明度を上げる
1856 TFT_draw_pic(&boss_img, boss.x, boss.y);
1857 // 白いオーバーレイを半透明風に描画
1858 for (int i = 0; i < h; i += 2) {
1859     for (int j = 0; j < w; j += 2) {
1860         int pos = (boss.y + i) * 320 + (boss.x + j);
1861         if (pos < 320*240) {
1862             framebuf[pos] = 0xFFFF; // 白いドット
1863         }
1864     }
1865 }
1866 } else {
1867     // 通常の描画
1868     TFT_draw_pic(&boss_img, boss.x, boss.y);
1869 }
1870 } else {
1871     // 画像が読み込まれていない場合は従来の矩形描画
1872     uint16_t draw_color = boss.color;
1873     if (boss.invincible_timer > 0 && (boss.invincible_timer % 4 < 2)) {
1874         draw_color = 0xFFFF; // 白く点滅
1875     }
1876     TFT_fill_rect(boss.x, boss.y, boss.width, boss.height, draw_color);
1877 }
1878
1879 // HPバーを描画（ボスの上部）
1880 int hp_bar_width = (boss.width * boss.hp) / boss.max_hp;
1881
1882 // HPバー背景（赤）
1883 TFT_fill_rect(boss.x, boss.y - 5, boss.width, 3, 0xF800);
1884
1885 // HPバー（緑→黄→赤と変化）
1886 if (hp_bar_width > 0) {
1887     uint16_t hp_color;
1888     if (boss.hp > boss.max_hp * 2 / 3) {
1889         hp_color = 0x07E0; // 緑
1890     } else if (boss.hp > boss.max_hp / 3) {
1891         hp_color = 0xFFE0; // 黄
1892     } else {

```



```

1893         hp_color = 0xF800; // 赤
1894     }
1895
1896     TFT_fill_rect(boss.x, boss.y - 5, hp_bar_width, 3, hp_color);
1897 }
1898 }
1899
1900 // ボスの弾発射
1901 void boss_shoot_bullets()
1902 {
1903     if (!boss.active) return;
1904
1905     boss.shoot_timer++;
1906
1907     // 弾幕パターン1: 直線発射
1908     if (boss.shoot_timer % 15 == 0) {
1909         spawn_enemy_bullet(boss.x, boss.y + boss.height/2);
1910     }
1911
1912     // 弾幕パターン2: 3方向発射
1913     if (boss.shoot_timer % 45 == 0) {
1914         spawn_enemy_bullet(boss.x, boss.y + boss.height/4);
1915         spawn_enemy_bullet(boss.x, boss.y + boss.height/2);
1916         spawn_enemy_bullet(boss.x, boss.y + boss.height*3/4);
1917     }
1918
1919     // 弾幕パターン3: 扇状発射 (HPが少ない時)
1920     if (boss.hp < boss.max_hp / 2 && boss.shoot_timer % 30 == 0) {
1921         for (int i = 0; i < 5; i++) {
1922             spawn_enemy_bullet(boss.x, boss.y + (boss.height * i) / 4);
1923         }
1924     }
1925 }
1926
1927 // プレイヤーの弾とボスの当たり判定
1928 void check_skill_boss_collision()
1929 {
1930     if (!boss.active || boss.invincible_timer > 0) return;
1931
1932     for (int i = 0; i < MAX_SKILLS; i++) {
1933         if (skills[i].active) {
1934             if (check_collision(skills[i].x, skills[i].y, skills[i].width, skills[i].
                height,
1935                                boss.x, boss.y, boss.width, boss.height)) {
1936                 // スキルとボスが当たった

```

```

1937         skills[i].active = 0;        // スキルを削除
1938         boss.hp--;                    // ボスのHPを1減らす
1939         boss.invincible_timer = 3;    // 3フレーム無敵
1940         score += 5;                  // ダメージスコア
1941
1942         // ウルト中のダメージカウント
1943         if (ult_active) {
1944             ult_damage_total++; // ウルト中のダメージをカウント
1945         }
1946
1947         break; // このスキルの処理を終了
1948     }
1949 }
1950 }
1951 }
1952
1953 // -----
1954 // ウルトシステム
1955 // -----
1956
1957 // ウルトゲージ更新
1958 void update_ult_gauge()
1959 {
1960     // 自動的にゲージが少しずつ減少（時間経過でゲージ減少）
1961     if (ult_gauge > 0 && !ult_active) {
1962         if (count % 300 == 0) { // 5秒に1回、1ずつ減少
1963             ult_gauge--;
1964         }
1965     }
1966 }
1967
1968 // ウルト発動
1969 void activate_ult()
1970 {
1971     if (ult_available && !ult_active) {
1972         ult_active = 1;
1973         ult_timer = ult_max_timer;
1974         ult_damage_total = 0;
1975         ult_gauge = 0; // ゲージをリセット
1976         ult_available = 0;
1977
1978         // ビーム発動
1979         ult_beam.active = 1;
1980         ult_beam.x = picx + 20; // プレイヤーの右端から
1981         ult_beam.y = picy;      // プレイヤーと同じ高さから

```

```

1982     ult_beam.width = 300;    // 画面右端まで
1983     ult_beam.height = 30;    // ビーム太さ
1984     ult_beam.color = 0x07FF; // シアン色
1985     ult_beam.damage_per_frame = 2; // フレーム当たりのダメージ
1986
1987     // ウルト発動エフェクト（画面全体に一瞬エフェクト）
1988     for (int i = 0; i < 320*120; i++) {
1989         framebuf[i] = 0x07FF; // シアン色で画面フラッシュ
1990     }
1991 }
1992 }
1993
1994 // ウルト状態更新
1995 void update_ult()
1996 {
1997     if (!ult_active) return;
1998
1999     ult_timer--;
2000
2001     // ビームの位置をプレイヤーに追従
2002     if (ult_beam.active) {
2003         ult_beam.x = picx + 20;
2004         ult_beam.y = picy;
2005         ult_beam.width = 300 - (picx + 20); // 画面右端までの幅を計算
2006         if (ult_beam.width < 0) ult_beam.width = 0;
2007     }
2008
2009     // ウルト終了判定
2010     if (ult_timer <= 0) {
2011         ult_active = 0;
2012         ult_timer = 0;
2013         ult_beam.active = 0; // ビーム停止
2014
2015         // ウルト終了時のボーナス判定（300ダメージ達成チェック）
2016         if (ult_damage_total >= 300) {
2017             score += 500; // 完璧ボーナス
2018         } else if (ult_damage_total >= 200) {
2019             score += 300; // 良いボーナス
2020         } else if (ult_damage_total >= 100) {
2021             score += 100; // 普通ボーナス
2022         }
2023
2024         ult_damage_total = 0;
2025     }
2026 }

```

```

2027 void TFT_fill_rect2(int x, int y, int width, int height, uint16_t color)
2028 {
2029     int i, j;
2030
2031     // 画面範囲チェック
2032     if (x < 0 || y < 0 || x >= 320 || y >= 240) return;
2033     if (x + width > 320) width = 320 - x;
2034     if (y + height > 240) height = 240 - y;
2035
2036     for (i = 0; i < height; i++)
2037     {
2038         for (j = 0; j < width; j++)
2039         {
2040             framebuf[(y + i) * 320 + (x + j)] = color;
2041         }
2042     }
2043 }
2044 // ウルトゲージ描画 (左下に配置)
2045 void draw_ult_gauge()
2046 {
2047     int gauge_x = 10;
2048     int gauge_y = 210;
2049     int gauge_width = 100;
2050     int gauge_height = 8;
2051
2052     // ゲージ背景 (黒)
2053     TFT_fill_rect2(gauge_x - 2, gauge_y - 2, gauge_width + 4, gauge_height + 4,
2054                   _COL_BLACK);
2055
2056     // ゲージ外枠 (白)
2057     TFT_fill_rect2(gauge_x - 1, gauge_y - 1, gauge_width + 2, gauge_height + 2,
2058                   _COL_WHITE);
2059
2060     // ゲージ内部 (グレー背景)
2061     TFT_fill_rect2(gauge_x, gauge_y, gauge_width, gauge_height, 0x8410);
2062
2063     // ゲージ本体
2064     int fill_width = (gauge_width * ult_gauge) / ult_max_gauge;
2065     // printf("fill_width:%d\n", fill_width);
2066     if (fill_width > 0) {
2067         uint16_t gauge_color;
2068         if (ult_available) {
2069             gauge_color = 0xFFE0; // 黄色 (使用可能)
2070         } else if (ult_gauge > 70) {
2071             gauge_color = 0x07E0; // 緑

```

```

2070     } else if (ult_gauge > 30) {
2071         gauge_color = 0xFD60; // オレンジ
2072     } else {
2073         gauge_color = 0xF800; // 赤
2074     }
2075
2076     TFT_fill_rect2(gauge_x, gauge_y, fill_width, gauge_height, gauge_color);
2077 }
2078
2079 // ウルトゲージラベル
2080 TFT_draw_string(gauge_x, gauge_y - 12, "ULT", _COL_BLACK);
2081
2082 // ウルト発動中のインジケーター
2083 if (ult_active) {
2084     TFT_draw_string(gauge_x + 40, gauge_y - 12, "ACTIVE!", 0xF800);
2085     // 残り時間表示
2086     int remaining_sec = (ult_timer / 60) + 1;
2087     TFT_draw_char(gauge_x + 80, gauge_y - 12, remaining_sec + '0', 0xF800);
2088 } else if (ult_available) {
2089     TFT_draw_string(gauge_x + 40, gauge_y - 12, "READY!", 0xFFE0);
2090 } else {
2091     TFT_fill_rect2(gauge_x + 40, gauge_y - 12, 60, 8, _COL_WHITE); // 空白で消す
2092 }
2093
2094 // ウルト中の総ダメージ表示
2095 if (ult_active && ult_damage_total > 0) {
2096     TFT_draw_string(gauge_x, gauge_y + 12, "DMG:", _COL_BLACK);
2097     TFT_draw_char(gauge_x + 24, gauge_y + 12, ((ult_damage_total/100) % 10) + '0',
2098                 , _COL_BLACK);
2099     TFT_draw_char(gauge_x + 30, gauge_y + 12, ((ult_damage_total/10) % 10) + '0',
2100                 , _COL_BLACK);
2101     TFT_draw_char(gauge_x + 36, gauge_y + 12, (ult_damage_total % 10) + '0',
2102                 , _COL_BLACK);
2103     TFT_draw_string(gauge_x + 42, gauge_y + 12, "/300", _COL_BLACK);
2104 }
2105 }
2106
2107 // ウルト入力チェック
2108 void check_ult_input()
2109 {
2110     // SW6ボタンでウルト発動
2111     static int sw6_pressed = 0;
2112
2113     if (SW6 == 1 && !sw6_pressed && ult_available) {
2114         activate_ult();
2115     }
2116 }

```

```

2112         sw6_pressed = 1;
2113     } else if (SW6 == 0) {
2114         sw6_pressed = 0;
2115     }
2116 }
2117
2118 // ビーム初期化
2119 void init_beam()
2120 {
2121     ult_beam.active = 0;
2122     ult_beam.x = 0;
2123     ult_beam.y = 0;
2124     ult_beam.width = 0;
2125     ult_beam.height = 30;
2126     ult_beam.color = 0x07FF; // シアン色
2127     ult_beam.damage_per_frame = 2;
2128 }
2129
2130 // ビーム描画
2131 void draw_beam()
2132 {
2133     static int green = 0x03E0;
2134     if (!ult_beam.active) return;
2135
2136     // ビーム本体を描画 (グラデーション効果)
2137     for (int i = 0; i < ult_beam.height; i++) {
2138         uint16_t beam_color;
2139
2140         // 中心部は明るく、端は暗くするグラデーション効果
2141         if (i < ult_beam.height / 4 || i >= ult_beam.height * 3 / 4) {
2142             beam_color = 0x0318; // 暗いシアン
2143         } else if (i < ult_beam.height / 2 || i >= ult_beam.height / 2) {
2144             beam_color = 0x05FF; // 中間シアン
2145         } else {
2146             beam_color = 0x07FF; // 明るいシアン
2147         }
2148
2149         // TFT_fill_rect(ult_beam.x, ult_beam.y + i, ult_beam.width, 1, beam_color);
2150         switch (i%4){
2151             case 0:
2152                 DMA0((void*)&beam_color, (void*)&framebuf[(ult_beam.y + i)*320 +
2153                     ult_beam.x], 1, 0, ult_beam.width);
2154             break;
2155             case 1:

```

```

2155         DMA1((void*)&beam_color, (void*)&framebuf[(ult_beam.y + i)*320 +
2156             ult_beam.x], 1, 0, ult_beam.width);
2157         break;
2158     case 2:
2159         DMA2((void*)&beam_color, (void*)&framebuf[(ult_beam.y + i)*320 +
2160             ult_beam.x], 1, 0, ult_beam.width);
2161         break;
2162     case 3:
2163         DMA3((void*)&beam_color, (void*)&framebuf[(ult_beam.y + i)*320 +
2164             ult_beam.x], 1, 0, ult_beam.width);
2165         break;
2166     }
2167 }
2168
2169 // ビームエフェクト (点滅)
2170 if ((count % 4) < 2) {
2171     // 外側のオーラ効果
2172     // TFT_fill_rect(ult_beam.x, ult_beam.y - 2, ult_beam.width, 1, 0x03E0); //
2173     // 薄い緑
2174     DMA2((void*)&green, (void*)&framebuf[(ult_beam.y - 2)*320 + ult_beam.x], 1, 0,
2175         ult_beam.width);
2176     // TFT_fill_rect(ult_beam.x, ult_beam.y + ult_beam.height + 1, ult_beam.width
2177     // , 1, 0x03E0);
2178     DMA3((void*)&green, (void*)&framebuf[(ult_beam.y + ult_beam.height + 1)*320 +
2179         ult_beam.x], 1, 0, ult_beam.width);
2180 }
2181 }
2182
2183 // ビームの当たり判定処理
2184 void check_beam_collision()
2185 {
2186     if (!ult_beam.active) return;
2187
2188     // 敵との当たり判定
2189     for (int i = 0; i < MAX_OBSTACLES; i++) {
2190         if (obstacles[i].active) {
2191             if (check_collision(ult_beam.x, ult_beam.y, ult_beam.width, ult_beam.
2192                 height,
2193                     obstacles[i].x, obstacles[i].y, obstacles[i].width,
2194                     obstacles[i].height)) {
2195                 // ビームと敵が当たった - 毎フレームダメージ
2196                 obstacles[i].hp -= ult_beam.damage_per_frame;
2197                 ult_damage_total += ult_beam.damage_per_frame; // ウルト中のダメージ
2198                 をカウント
2199             }
2200         }
2201     }
2202 }

```

```

2190         if (obstacles[i].hp <= 0) {
2191             obstacles[i].active = 0; // HPが0になったら敵を削除
2192             score += 20; // ビームで倒したボーナス
2193         } else {
2194             score += ult_beam.damage_per_frame; // ダメージスコア
2195         }
2196     }
2197 }
2198 }
2199
2200 // ボスとの当たり判定
2201 if (boss.active) {
2202     if (check_collision(ult_beam.x, ult_beam.y, ult_beam.width, ult_beam.height,
2203                         boss.x, boss.y, boss.width, boss.height)) {
2204         // ビームとボスが当たった - 毎フレームダメージ
2205         boss.hp -= ult_beam.damage_per_frame;
2206         boss.invincible_timer = 1; // 1フレーム無敵 (ビームは連続ダメージ)
2207         ult_damage_total += ult_beam.damage_per_frame; // ウルト中のダメージをカ
            ウント
2208         score += ult_beam.damage_per_frame; // ダメージスコア
2209     }
2210 }
2211 }
2212 // -----
2213 // MIDI変換関数群
2214 // -----
2215 // MIDIノート番号を周波数に変換
2216 uint16_t midi_note_to_freq(uint8_t note)
2217 {
2218     if (note >= 128) return 0;
2219     return midi_freq_table[note];
2220 }
2221
2222 // 可変長数値の読み取り
2223 uint32_t read_variable_length(uint8_t *data, int *offset)
2224 {
2225     uint32_t value = 0;
2226     uint8_t byte;
2227
2228     do {
2229         byte = data[(*offset)++];
2230         value = (value << 7) | (byte & 0x7F);
2231     } while (byte & 0x80);
2232
2233     return value;

```



```

2234 }
2235
2236 // MIDIトラックデータを解析してTONE配列に変換
2237 int process_midi_track(uint8_t *track_data, int track_length, struct TONE *tone_array
    )
2238 {
2239     int offset = 0;
2240     int tone_count = 0;
2241     uint32_t current_time = 0;
2242     uint32_t last_note_end_time = 0; // 最後の音符が終わった時刻
2243     uint8_t running_status = 0;
2244     int event_count = 0;
2245
2246     // アクティブなノートを追跡する配列
2247     uint8_t active_notes[128];
2248     uint32_t note_start_time[128];
2249     int i;
2250
2251     // 初期化
2252     for (i = 0; i < 128; i++) {
2253         active_notes[i] = 0;
2254         note_start_time[i] = 0;
2255     }
2256
2257     while (offset < track_length && tone_count < MAX_MIDI_EVENTS - 1) {
2258         event_count++;
2259
2260         // デルタタイム読み取り
2261         int delta_offset_start = offset;
2262         uint32_t delta_time = read_variable_length(track_data, &offset);
2263         current_time += delta_time;
2264
2265         if (offset >= track_length) {
2266             break;
2267         }
2268
2269         // イベントタイプ読み取り
2270         uint8_t event_byte = track_data[offset];
2271
2272         // ランニングステータスの処理
2273         if (event_byte & 0x80) {
2274             running_status = event_byte;
2275             offset++;
2276         } else {
2277             // ランニングステータスを使用

```

```

2278         event_byte = running_status;
2279     }
2280
2281     if (offset >= track_length) {
2282         break;
2283     }
2284
2285     // ノートオンイベント
2286     if ((event_byte & 0xF0) == MIDI_NOTE_ON) {
2287         if (offset + 1 >= track_length) break;
2288         uint8_t note = track_data[offset++];
2289         uint8_t velocity = track_data[offset++];
2290
2291         if (velocity > 0 && note < 128) {
2292             active_notes[note] = 1;
2293             note_start_time[note] = current_time;
2294         } else if (note < 128) {
2295             // ベロシティ0はノートオフと同じ
2296             if (active_notes[note]) {
2297                 active_notes[note] = 0;
2298                 uint32_t duration = current_time - note_start_time[note];
2299                 uint32_t note_end_time = current_time;
2300
2301                 // 前の音符との間に休符があるかチェック
2302                 if (last_note_end_time > 0 && note_start_time[note] >
2303                     last_note_end_time) {
2304                     uint32_t rest_duration = note_start_time[note] -
2305                         last_note_end_time;
2306                     uint32_t rest_ms = (rest_duration * 1000) / TICKS_PER_QUARTER
2307                         ;
2308
2309                     if (rest_ms > 50 && tone_count < MAX_MIDI_EVENTS - 1) { //
2310                         50ms以上の休符のみ
2311                         tone_array[tone_count].frequency = 0; // 無音
2312                         tone_array[tone_count].duration = rest_ms;
2313                         tone_count++;
2314                     }
2315                 }
2316
2317                 if (tone_count < MAX_MIDI_EVENTS) {
2318                     tone_array[tone_count].frequency = midi_note_to_freq(note);
2319                     tone_array[tone_count].duration = (duration * 1000) /
2320                         TICKS_PER_QUARTER; // ms変換
2321                     tone_count++;

```

```

2317         last_note_end_time = note_end_time;  // 最後の音符終了時刻を
           更新
2318     }
2319 }
2320 }
2321 }
2322 // ノートオフイベント
2323 else if ((event_byte & 0xF0) == MIDI_NOTE_OFF) {
2324     if (offset + 1 >= track_length) break;
2325     uint8_t note = track_data[offset++];
2326     uint8_t velocity = track_data[offset++];
2327
2328     if (note < 128 && active_notes[note]) {
2329         active_notes[note] = 0;
2330         uint32_t duration = current_time - note_start_time[note];
2331         uint32_t note_end_time = current_time;
2332
2333         // 前の音符との間に休符があるかチェック
2334         if (last_note_end_time > 0 && note_start_time[note] >
            last_note_end_time) {
2335             uint32_t rest_duration = note_start_time[note] -
                last_note_end_time;
2336             uint32_t rest_ms = (rest_duration * 1000) / TICKS_PER_QUARTER;
2337
2338             if (rest_ms > 50 && tone_count < MAX_MIDI_EVENTS - 1) {  // 50ms
                以上の休符のみ
2339                 tone_array[tone_count].frequency = 0;  // 無音
2340                 tone_array[tone_count].duration = rest_ms;
2341                 tone_count++;
2342             }
2343         }
2344
2345         if (tone_count < MAX_MIDI_EVENTS) {
2346             tone_array[tone_count].frequency = midi_note_to_freq(note);
2347             tone_array[tone_count].duration = (duration * 1000) /
                TICKS_PER_QUARTER;  // ms変換
2348             tone_count++;
2349             last_note_end_time = note_end_time;  // 最後の音符終了時刻を更新
2350         }
2351     }
2352 }
2353 // その他のイベント（スキップ）
2354 else if ((event_byte & 0xF0) == 0xC0 || (event_byte & 0xF0) == 0xD0) {
2355     // プログラムチェンジ、チャンネルプレッシャー（1バイト）
2356     if (offset < track_length) offset++;

```

```

2357     }
2358     else if ((event_byte & 0xF0) >= 0x80 && (event_byte & 0xF0) <= 0xE0) {
2359         // その他の2バイトメッセージ
2360         if (offset + 1 < track_length) offset += 2;
2361     }
2362     else if (event_byte == 0xFF) {
2363         // メタイベント
2364         if (offset >= track_length) break;
2365         uint8_t meta_type = track_data[offset++];
2366         uint32_t length = read_variable_length(track_data, &offset);
2367
2368         if (meta_type == 0x2F) {
2369             // End of Track
2370             break;
2371         }
2372
2373         offset += length;
2374     }
2375     else if (event_byte == 0xF0 || event_byte == 0xF7) {
2376         // システムエクスクルーシブ
2377         uint32_t length = read_variable_length(track_data, &offset);
2378         offset += length;
2379     }
2380 }
2381
2382 return tone_count;
2383 }
2384
2385 // MIDIファイルをTONE配列に変換するメイン関数
2386 int parse_midi_to_tone(struct file_t *midi_file, struct TONE *tone_array)
2387 {
2388     uint8_t *data = (uint8_t*)midi_file->Data;
2389
2390     // 最小ファイルサイズチェック
2391     if (midi_file->FileSize < 14) {
2392         return 0;
2393     }
2394
2395     // MIDIヘッダーの確認
2396     if (data[0] != 'M' || data[1] != 'T' || data[2] != 'h' || data[3] != 'd') {
2397         return 0;
2398     }
2399
2400     // ヘッダー長の確認 (通常は6)

```

```

2401     uint32_t header_length = (data[4] << 24) | (data[5] << 16) | (data[6] << 8) |
        data[7];
2402
2403     // ヘッダー情報の読み取り (ビッグエンディアン)
2404     uint16_t format_type = (data[8] << 8) | data[9];
2405     uint16_t track_count = (data[10] << 8) | data[11];
2406     uint16_t time_division = (data[12] << 8) | data[13];
2407
2408     // 値の妥当性チェック
2409     if (format_type > 2) {
2410         return 0;
2411     }
2412     if (track_count > 100) {
2413         return 0;
2414     }
2415     if (time_division == 0 || time_division > 32767) {
2416         return 0;
2417     }
2418
2419     int offset = 14; // ヘッダー後の位置
2420     int total_tones = 0;
2421
2422     // 全てのトラックを処理
2423     for (int track_num = 0; track_num < track_count && track_num < 2; track_num++) {
2424
2425         if (offset + 8 >= midi_file->FileSize) {
2426             break;
2427         }
2428
2429         // トラックヘッダーの確認
2430         if (data[offset] == 'M' && data[offset+1] == 'T' &&
2431             data[offset+2] == 'r' && data[offset+3] == 'k') {
2432
2433             uint32_t track_length = (data[offset+4] << 24) | (data[offset+5] << 16) |
2434                 (data[offset+6] << 8) | data[offset+7];
2435
2436             offset += 8; // トラックヘッダーをスキップ
2437
2438             if (offset + track_length <= midi_file->FileSize) {
2439                 int track_tones = process_midi_track(&data[offset], track_length,
2440                     &tone_array[total_tones]);
2441                 total_tones += track_tones;
2442                 offset += track_length; // 次のトラックへ
2443             } else {
2444                 break;

```

```

2445         }
2446     } else {
2447         break;
2448     }
2449 }
2450
2451     return total_tones;
2452 }
2453 // -----
2454 // DMA関数群
2455 // -----
2456 void init_DMAL(void){
2457     STB.CR2.BIT._DMAC = 0; // 0で有効化
2458     DMAC.DMAOR.BIT.DME = 1; //DMAC有効化
2459     //DMAC0
2460     DMAC0.CHCR.BIT.RS = 4; //リソースセレクト4
2461     DMAC0.CHCR.BIT.TS = 1; //16bit転送
2462     DMAC0.CHCR.BIT.TB = 0; //サイクルモード
2463     DMAC0.CHCR.BIT.IE = 0; //割り込み禁止
2464     //DMAC1
2465     DMAC1.CHCR.BIT.RS = 4; //リソースセレクト4
2466     DMAC1.CHCR.BIT.TS = 1; //16bit転送
2467     DMAC1.CHCR.BIT.TB = 0; //サイクルモード
2468     DMAC1.CHCR.BIT.IE = 0; //割り込み禁止
2469     //DMAC2
2470     DMAC2.CHCR.BIT.RS = 4; //リソースセレクト4
2471     DMAC2.CHCR.BIT.TS = 1; //16bit転送
2472     DMAC2.CHCR.BIT.TB = 0; //サイクルモード
2473     DMAC2.CHCR.BIT.IE = 0; //割り込み禁止
2474     //DMAC3
2475     DMAC3.CHCR.BIT.RS = 4; //リソースセレクト4
2476     DMAC3.CHCR.BIT.TS = 1; //16bit転送
2477     DMAC3.CHCR.BIT.TB = 0; //サイクルモード
2478     DMAC3.CHCR.BIT.IE = 0; //割り込み禁止
2479
2480 }
2481
2482 void TFT_clear2(void){
2483     for(int i = 0;i < 240*320;i++){
2484         framebuf[i] = _COL_WHITE;
2485     }
2486 }
2487 /**
2488  * @brief DMAC0設定
2489  *

```

```

2490  * @param DM デスティネーションアドレス. 転送先 0→固定,1→インクリメント
2491  * @param SM ソースアドレス. 転送元. 転送先 0→固定,1→インクリメント
2492  * @param count 転送回数
2493  */
2494 void DMA0(void *SAR, void *DAR, uint8_t DM, uint8_t SM, int count){
2495     // DMAC0動作停止
2496     if(DMAC0f){
2497         while(DMAC0.CHCR.BIT.TE == 0)
2498             ;
2499         DMAC0.CHCR.BIT.DE = 0;
2500
2501     }else{
2502         DMAC0f=1;
2503     }
2504     // DMAC0設定
2505     DMAC0.SAR = (void*)SAR;    // ソースアドレス
2506     DMAC0.DAR = (void *)DAR;    // デスティネーションアドレス
2507     DMAC0.DMATCR = count - 1; // 転送カウント(16bit単位)
2508     //CHCRの設定
2509     DMAC0.CHCR.BIT.DM = DM;    //デスティネーションアドレス固定
2510     DMAC0.CHCR.BIT.SM = SM;    //ソースアドレスインクリメント
2511
2512     DMAC0.CHCR.BIT.TE = 0;    //転送終了フラグクリア
2513     DMAC0.CHCR.BIT.DE = 1;    //DMA有効化
2514 }
2515 /**
2516  * @brief DMAC1設定
2517  *
2518  * @param DM デスティネーションアドレス. 転送先 0→固定,1→インクリメント
2519  * @param SM ソースアドレス. 転送元. 転送先 0→固定,1→インクリメント
2520  * @param count 転送回数
2521  */
2522 void DMA1(void *SAR, void *DAR, uint8_t DM, uint8_t SM, int count){
2523     // DMAC0動作停止
2524     if(DMAC1f){
2525         while(DMAC1.CHCR.BIT.TE == 0)
2526             ;
2527         DMAC1.CHCR.BIT.DE = 0;
2528
2529     }else{
2530         DMAC1f=1;
2531     }
2532     // DMAC0設定
2533     DMAC1.SAR = (void*)SAR;    // ソースアドレス
2534     DMAC1.DAR = (void *)DAR;    // デスティネーションアドレス

```

```

2535     DMAC1.DMATCR = count - 1; // 転送カウント(16bit単位)
2536     //CHCRの設定
2537     DMAC1.CHCR.BIT.DM = DM; //デスティネーションアドレス固定
2538     DMAC1.CHCR.BIT.SM = SM; //ソースアドレスインクリメント
2539
2540     DMAC1.CHCR.BIT.TE = 0; //転送終了フラグクリア
2541     DMAC1.CHCR.BIT.DE = 1; //DMA有効化
2542 }
2543 /**
2544  * @brief DMAC2設定
2545  *
2546  * @param DM デスティネーションアドレス. 転送先 0→固定,1→インクリメント
2547  * @param SM ソースアドレス. 転送元. 転送先 0→固定,1→インクリメント
2548  * @param count 転送回数
2549  */
2550 void DMA2(void *SAR, void *DAR, uint8_t DM, uint8_t SM, int count){
2551     // DMAC0動作停止
2552     if(DMAC2f){
2553         while(DMAC2.CHCR.BIT.TE == 0)
2554             ;
2555         DMAC2.CHCR.BIT.DE = 0;
2556
2557     }else{
2558         DMAC2f=1;
2559     }
2560     // DMAC0設定
2561     DMAC2.SAR = (void*)SAR; // ソースアドレス
2562     DMAC2.DAR = (void *)DAR; // デスティネーションアドレス
2563     DMAC2.DMATCR = count - 1; // 転送カウント(16bit単位)
2564     //CHCRの設定
2565     DMAC2.CHCR.BIT.DM = DM; //デスティネーションアドレス固定
2566     DMAC2.CHCR.BIT.SM = SM; //ソースアドレスインクリメント
2567
2568     DMAC2.CHCR.BIT.TE = 0; //転送終了フラグクリア
2569     DMAC2.CHCR.BIT.DE = 1; //DMA有効化
2570 }
2571 /**
2572  * @brief DMAC3設定
2573  *
2574  * @param DM デスティネーションアドレス. 転送先 0→固定,1→インクリメント
2575  * @param SM ソースアドレス. 転送元. 転送先 0→固定,1→インクリメント
2576  * @param count 転送回数
2577  */
2578 void DMA3(void *SAR, void *DAR, uint8_t DM, uint8_t SM, int count){
2579     // DMAC0動作停止

```



```

2580     if(DMAC3f){
2581         while(DMAC3.CHCR.BIT.TE == 0)
2582             ;
2583         DMAC3.CHCR.BIT.DE = 0;
2584
2585     }else{
2586         DMAC3f=1;
2587     }
2588     // DMAC0設定
2589     DMAC3.SAR = (void*)SAR;    // ソースアドレス
2590     DMAC3.DAR = (void *)DAR;    // デスティネーションアドレス
2591     DMAC3.DMATCR = count - 1; // 転送カウント(16bit単位)
2592     //CHCRの設定
2593     DMAC3.CHCR.BIT.DM = DM;    //デスティネーションアドレス固定
2594     DMAC3.CHCR.BIT.SM = SM;    //ソースアドレスインクリメント
2595
2596     DMAC3.CHCR.BIT.TE = 0;    //転送終了フラグクリア
2597     DMAC3.CHCR.BIT.DE = 1;    //DMA有効化
2598 }
2599
2600
2601 void LCD_init(void)
2602 {
2603     wait_us(45000);
2604     LCD_inst(0x30);
2605     wait_us(4100);
2606     LCD_inst(0x30);
2607     wait_us(100);
2608     LCD_inst(0x30);
2609
2610     LCD_inst(0x38);
2611     LCD_inst(0x08);
2612     LCD_inst(0x01);
2613     wait_us(1640);
2614     LCD_inst(0x06);
2615     LCD_inst(0x0c);
2616 }
2617 void LCD_cursor(_UINT x, _UINT y)
2618 {
2619     if (x > 15)
2620         x = 15;
2621     if (y > 1)
2622         y = 1;
2623     LCD_inst(0x80 | x | y << 6);
2624 }

```

```

2625
2626 void LCD_putchar(_SBYTE ch)
2627 {
2628     LCD_data(ch);
2629 }
2630
2631 void LCD_putstr(_SBYTE *str)
2632 {
2633     _SBYTE ch;
2634
2635     while (ch = *str++)
2636         LCD_putchar(ch);
2637 }
2638 void LCD_inst(_SBYTE inst)
2639 {
2640     LCD_E = 0;
2641     LCD_RS = 0;
2642     LCD_RW = 0;
2643     LCD_E = 1;
2644     LCD_DATA = inst;
2645     wait_us(1);
2646     LCD_E = 0;
2647     wait_us(40);
2648 }
2649
2650 void LCD_data(_SBYTE data)
2651 {
2652     LCD_E = 0;
2653     LCD_RS = 1;
2654     LCD_RW = 0;
2655     LCD_E = 1;
2656     LCD_DATA = data;
2657     wait_us(1);
2658     LCD_E = 0;
2659     wait_us(40);
2660 }
2661 // -----
2662 void main()
2663 {
2664     int i;
2665     int j;
2666     PFC.PAIORH.BYTE.L |= 0x0F;
2667     PFC.PEIORL.BIT.B3 = 1;    // 1 の位
2668     PFC.PEIORL.BIT.B2 = 1;    // 10 の位
2669     PFC.PEIORL.BIT.B1 = 1;    // 100 の位

```

```

2670     printf("initializing...\n");
2671     init_MTU2(); //ADC
2672     printf("MTU2 initialized.\n");
2673     init_CMT1(); //FPS counter
2674     printf("CMT1 initialized.\n");
2675     // while(1)
2676     // ;
2677     init_CMT0(); //wait_us
2678     printf("CMT0 initialized.\n");
2679     init_SCI2();
2680     printf("SCI2 initialized.\n");
2681     init_CS2();
2682     printf("CS2 initialized.\n");
2683     init_DMAL();
2684     printf("DMAL initialized.\n");
2685     LCD_init();
2686     printf("LCD initialized.\n");
2687     TFT_On();
2688     printf("TFT On.\n");
2689     // TFT_clear();
2690     TFT_clear2();
2691     PFC.PEIORL.BIT.BO = 1;
2692     DMAL.DMALR.BIT.DME = 1; //DMAL有効化
2693
2694     DMAL0f = 0;
2695     DMAL1f = 0;
2696     DMAL2f = 0;
2697     DMAL3f = 0;
2698
2699     if (0)
2700         printf("No card found\n");
2701     else
2702     {
2703         printf("Card found\n");
2704         if (Enter_SPI_mode() < 0){
2705             printf("SPI mode Err\n");
2706             TFT_draw_string(0, 0, "SPI mode Err", _COL_BLACK);
2707             TFTCTRL = 0x4001;
2708             DMA0((void *)framebuf, (void*) 0x08000000, 0, 1, 320 * 240);
2709         }
2710         else
2711         {
2712             TFT_draw_string(0, 0, "Loading SD Card...", _COL_BLACK);
2713             TFTCTRL = 0x4001;
2714             DMA0((void *)framebuf, (void*) 0x08000000, 0, 1, 320 * 240);

```

```

2715     printf("SPI mode\n");
2716
2717     SD_send_cmd(0x50, 512); // ブロックサイズ=512
2718     ex_MBR();
2719     ex_BPB();
2720     get_FAT();
2721     get_RDE(0);
2722     mari.n = 3; // RDE 5 番目のファイル情報
2723     mari.Data = FileData0;
2724     show_all_files();
2725     get_RDE(0);
2726     printf("Loading files...\n");
2727
2728     // 自動ファイル読み込み関数を使用
2729     int loaded_files = load_files_by_name();
2730
2731     if (loaded_files == 0) {
2732         printf("No files could be loaded! Check file names.\n");
2733     }
2734     // TFT_draw_pic(&buttle,picx,picy);
2735     // printf("showing2\n");
2736     uint16_t white = _COL_WHITE;
2737     // TFT_clear2();
2738     // printf("showing\n");
2739     // while(DMAC2.CHCR.BIT.TE == 0)
2740     //     ;
2741     // while(1);
2742
2743
2744     // .IMGファイル用の簡略化されたサイズ取得
2745     printf("loaded\n");
2746     MTU2.TSTR.BIT.CST3 = 1; // MTU2 CH3スタート
2747     music_playing = 0;
2748     // TFT_draw_pic_dma(framebuf,FileData3 + 1,320*240); // 背景クリア
2749     TFT_clear2();
2750     DMA0((void *)FileData3,(void*) framebuf,1,1,320 * 120);
2751     // while(DMAC0.CHCR.BIT.TE == 0);
2752     DMA1((void *)(&FileData3[320*120]),(void*) (&framebuf
        [320*120]),1,1,320 * 120);
2753     while(DMAC1.CHCR.BIT.TE == 0 || DMAC0.CHCR.BIT.TE == 0)
2754         ;
2755     TFTCTRL = 0x4001;
2756     DMA0((void *)framebuf,(void*) 0x08000000,0,1,320 * 240);
2757     LCD_cursor(0, 0);
2758     LCD_putstr("shooting game");

```

```

2759     LCD_cursor(0, 1);
2760     LCD_putstr("SW4 to start");
2761     MTU2.TSTR.BIT.CST4 = 1;           // MTU2 CH4スタート
2762     while(SW4 == 0)
2763     ;
2764     LCD_cursor(0, 1);
2765     LCD_putstr("                ");
2766     TFT_draw_string(0,200,"Loading...",_COL_BLACK);
2767     TFTCTRL = 0x4001;
2768     DMA0((void *)framebuf,(void*) 0x08000000,0,1,320 * 240);
2769     buttle.Data = FileData3;
2770     if (find_and_load_file(&buttle, "BUTTLE .IMG")) {
2771         printf("BUTTLE.IMG loaded\n");
2772     } else {
2773         printf("BUTTLE.IMG not found\n");
2774     }
2775     MTU2.TSTR.BIT.CST0 = 1;           // MTU2 CH0スタート //ADC
2776     CMT.CMSTR.BIT.STR1 = 1;          // CMT1スタート //1秒間隔
2777
2778     // -----
2779     while(1){
2780         uint8_t *raw_mari = (uint8_t*)mari.Data;
2781         int w = raw_mari[0] + (raw_mari[1] << 8); // width (1バイト目)
2782         int h = raw_mari[3]; // height (2バイト目)
2783         int hue = 0;
2784         while(SW4 == 1)
2785         ;
2786
2787
2788         LCD_cursor(0, 0);
2789         LCD_putstr("shooting game");
2790         LCD_cursor(0, 1);
2791         LCD_putstr("SW4 to end game");
2792         // BUTTLE.IMGを読み込み
2793         TFT_clear2();
2794         init_obstacles(); // 敵初期化
2795         init_skills(); // プレイヤーの弾初期化
2796         init_enemy_bullets(); // 敵の弾初期化
2797         init_boss(); // ボス初期化
2798         init_beam(); // ビーム初期化
2799         uint16_t cd = 0;
2800
2801         // プレイヤーを画面左端に配置
2802         picx = 10;
2803         picy = 60;

```

```

2804         j = 0;
2805         music_playing = 1;
2806         timing = 0;
2807         while (1){
2808             DMA1((void *)FileData3,(void*) framebuf,1,1,320 * 60);
2809             DMA2((void *)&FileData3[320*60]),(void*) (&framebuf
                [320*60]),1,1,320 * 60);
2810             DMA3((void *)&FileData3[320*120]),(void*) (&framebuf
                [320*120]),1,1,320 * 60);
2811             // printf("ジョイスティック\n");
2812             // ジョイスティック制御（横型シューティング用）
2813             if ((AD0.ADDR0 >> 6) < 400)
2814             {
2815                 // -- ジョイスティック上 --
2816                 picy -= (20 - (AD0.ADDR0 >> 6) / 20) + 1;
2817                 // 画像の高さを考慮した境界設定
2818                 if(picy <= 0){
2819                     picy = 0;
2820                 }
2821             }
2822             else if ((AD0.ADDR0 >> 6) > 600)
2823             {
2824                 // -- ジョイスティック下 --
2825                 picy += ((AD0.ADDR0 >> 6) - 600) / 20 + 1;
2826                 // 画像の高さを考慮した境界設定
2827                 if(picy >= (180 - h)){
2828                     picy = 180 - h;
2829                 }
2830             }
2831             if ((AD0.ADDR1 >> 6) < 400)
2832             {
2833                 // -- ジョイスティック右 --
2834                 picx += (20 - (AD0.ADDR1 >> 6) / 20) + 1;
2835                 // 画面中央付近まで移動可能
2836                 if(picx >= 160){
2837                     picx = 160;
2838                 }
2839             }
2840             else if ((AD0.ADDR1 >> 6) > 600)
2841             {
2842                 // -- ジョイスティック左 --
2843                 picx -= ((AD0.ADDR1 >> 6) - 600) / 20 + 1;
2844                 if(picx <= 0){
2845                     picx = 0;
2846                 }

```

```

2847     }
2848
2849     // プレイヤーの弾発射 (SW5ボタンで連射)
2850     if (SW5 == 1 && cd >= 5) // 連射間隔を短く
2851     {
2852         spawn_skill(picx, picy, w, h);
2853         cd = 0;
2854     }
2855     cd++;
2856     // printf("ウルトシステム\n");
2857     // ウルトシステム更新
2858     update_ult_gauge(); // ゲージ自然減少
2859     check_ult_input(); // ウルト発動入力チェック
2860     update_ult(); // ウルト状態更新
2861
2862     // TFT_draw_pic_dma(framebuf, FileData1, 320*120); // 背景クリア
2863     // DMA1((void *)FileData1, (void*) framebuf, 1, 1, 320 * 120);
2864     // printf("画面描画\n");
2865     // printf("\n");
2866     // DMA1((void *)FileData3, (void*) framebuf + 320*120, 1, 1, 320 *
        120);
2867     // while(DMAC1.CHCR.BIT.TE == 0)
2868     //     ;
2869
2870
2871     // ビームシステム
2872     check_beam_collision(); // ビームの当たり判定
2873
2874     // 敵システム
2875     // printf("enem\n");
2876     update_obstacles(); // 敵更新
2877
2878     // プレイヤーの弾システム
2879     // printf("skill\n");
2880     update_skills(); // プレイヤーの弾更新
2881     check_skill_collision(); // プレイヤーの弾と敵の当たり判定
2882
2883     // 敵の弾幕システム
2884     // printf("enem_skill\n");
2885     // printf("1\n");
2886     update_enemy_bullets(); // 敵の弾更新
2887     // printf("2\n");
2888     check_player_bullet_collision(picx, picy, w, h); // プレイヤーと
        敵の弾の当たり判定
2889     // printf("3\n");

```

```

2890
2891 // ボスシステム
2892 // printf("boss\n");
2893 spawn_boss(); // ボス生成チェック
2894 update_boss(); // ボス更新
2895 check_skill_boss_collision(); // プレイヤーの弾とボスの当たり判定
2896
2897 // プレイヤーと敵の当たり判定
2898 check_player_collision(picx, picy, w, h);
2899
2900
2901 // UI表示
2902 // プレイヤー描画
2903 while(DMAC1.CHCR.BIT.TE == 0 || DMAC2.CHCR.BIT.TE == 0 || DMAC3.
    CHCR.BIT.TE == 0){
2904     // printf(".");
2905 }
2906 draw_beam(); // ビーム描画
2907 TFT_draw_pic(&mari,picx,picy);
2908 draw_obstacles(); // 敵描画
2909 draw_skills(); // プレイヤーの弾描画
2910 draw_enemy_bullets(); // 敵の弾描画
2911 draw_boss(); // ボス描画
2912 TFT_draw_char(0,0,(lastcount/10)%10 + '0',_COL_BLACK);
2913 TFT_draw_char(6,0,lastcount%10 + '0',_COL_BLACK);
2914 TFT_draw_string(12,0,"FPS",_COL_BLACK);
2915 while(DMAC1.CHCR.BIT.TE == 0 || DMAC2.CHCR.BIT.TE == 0 || DMAC3.
    CHCR.BIT.TE == 0){
2916     // printf(".");
2917 }
2918
2919 if(frag){
2920     frag = 0;
2921
2922     // スコア表示 (120px以下に配置)
2923     TFT_draw_string(10,190,"SCORE:",_COL_BLACK);
2924     TFT_draw_char(46,190,((score/1000)%10) + '0',_COL_BLACK);
2925     TFT_draw_char(52,190,((score/100)%10) + '0',_COL_BLACK);
2926     TFT_draw_char(58,190,((score/10)%10) + '0',_COL_BLACK);
2927     TFT_draw_char(64,190,(score%10) + '0',_COL_BLACK);
2928
2929     // ダメージ表示 (120px以下に配置)
2930     TFT_draw_string(90,190,"DMG:",_COL_BLACK);
2931     TFT_draw_char(116,190,(collision_count/10)%10 + '0',
        _COL_BLACK);

```



```

2932     TFT_draw_char(122,190,collision_count%10 + '0',_COL_BLACK);
2933
2934     // フィールド名表示
2935     TFT_draw_string(160,230,field_data[current_field].name,
        _COL_BLACK);
2936
2937
2938     // ボスHP表示（ボスが出現している時のみ）
2939     if (boss.active) {
2940         TFT_draw_string(180,190,"BOSS:",_COL_BLACK);
2941         TFT_draw_char(216,190,((boss.hp/1000)%10) + '0',
        _COL_BLACK);
2942         TFT_draw_char(222,190,((boss.hp/100)%10) + '0',_COL_BLACK
        );
2943         TFT_draw_char(228,190,((boss.hp/10)%10) + '0',_COL_BLACK
        );
2944         TFT_draw_char(234,190,(boss.hp%10) + '0',_COL_BLACK);
2945     }
2946
2947     // ウルトゲージ描画（常に表示）
2948     draw_ult_gauge();
2949     TFTCTRL = 0x4001;
2950     DMA0((void *)framebuf,(void*) 0x08000000,0,1,320 * 240);
2951 }else{
2952     TFTCTRL = 0x4001;
2953     DMA0((void *)framebuf,(void*) 0x08000000,0,1,320 * 180);
2954 }
2955
2956
2957
2958     count++;
2959     if(timing_frag){
2960         timing_frag = 0;
2961     }
2962     if(SW4 == 1){
2963         while(SW4 == 1)
2964             ;
2965         TFT_fill_rect2(80, 40, 160, 80, _COL_WHITE);
2966         TFT_draw_string(90,50,"SW5 to restart",_COL_BLACK);
2967         TFT_draw_string(90,60,"SW6 to return game",_COL_BLACK);
2968         TFTCTRL = 0x4001;
2969         DMA0((void *)framebuf,(void*) 0x08000000,0,1,320 * 180);
2970         while(SW6 == 0){
2971             if(SW5 == 1){
2972                 while(SW5 == 1)

```

```

2973         ;
2974         TFT_clear2();
2975         init_obstacles(); // 敵初期化
2976         init_skills(); // プレイヤーの弾初期化
2977         init_enemy_bullets(); // 敵の弾初期化
2978         init_boss(); // ボス初期化
2979         init_beam(); // ビーム初期化
2980         cd = 0;
2981
2982         // プレイヤーを画面左端に配置
2983         picx = 10;
2984         picy = 60;
2985         j = 0;
2986         music_playing = 1;
2987         timing = 0;
2988         score = 0;
2989         ult_gauge = 0;
2990         ult_active = 0;
2991         break; // フィールド選択へ戻る
2992     }
2993 }
2994 while(SW6==1);
2995 }
2996 }
2997 }
2998 }
2999 }
3000
3001 SD_CS = 1; // CS negate
3002
3003 while (1)
3004     ;
3005 }
3006
3007 // -----
3008 // TONE配列からTIMER_DATA配列への変換関数
3009 // MTU2のタイマー値を事前計算して格納
3010 void convert_tone_to_timer_data(struct TONE *tone_array, int count, struct TIMER_DATA
    *timer_data)
3011 {
3012     int i;
3013
3014     for (i = 0; i < count; i++) {
3015         uint16_t frequency = tone_array[i].frequency;
3016         uint16_t duration = tone_array[i].duration;

```

```

3017
3018     if (frequency > 0) {
3019         // 音ありの場合
3020         timer_data[i].is_sound = 1;
3021
3022         // MTU21用の周波数設定値を事前計算
3023         // 周波数からTGRA値を計算: (31250 / frequency) * 5 - 1
3024         uint16_t timer_count = (31250 / frequency) * 5 - 1;
3025         timer_data[i].tgra_sound = timer_count;
3026
3027         // MTU23用の待機時間設定値を事前計算
3028         // 持続時間からTGRA値を計算: 31250 / (2000 / duration) - 1
3029         uint32_t wait_count = 31250 / (2000 / duration) - 1;
3030         wait_count = wait_count > 65535 ? 65535 : wait_count;
3031         timer_data[i].tgra_wait = (uint16_t)wait_count;
3032     } else {
3033         // 無音の場合
3034         timer_data[i].is_sound = 0;
3035         timer_data[i].tgra_sound = 0; // 使用しない
3036
3037         // MTU23用の待機時間設定値のみ計算
3038         uint32_t wait_count = 31250 / (2000 / duration) - 1;
3039         wait_count = wait_count > 65535 ? 65535 : wait_count;
3040         timer_data[i].tgra_wait = (uint16_t)wait_count;
3041     }
3042 }
3043 }
3044 // -----

```
